

universität freiburg

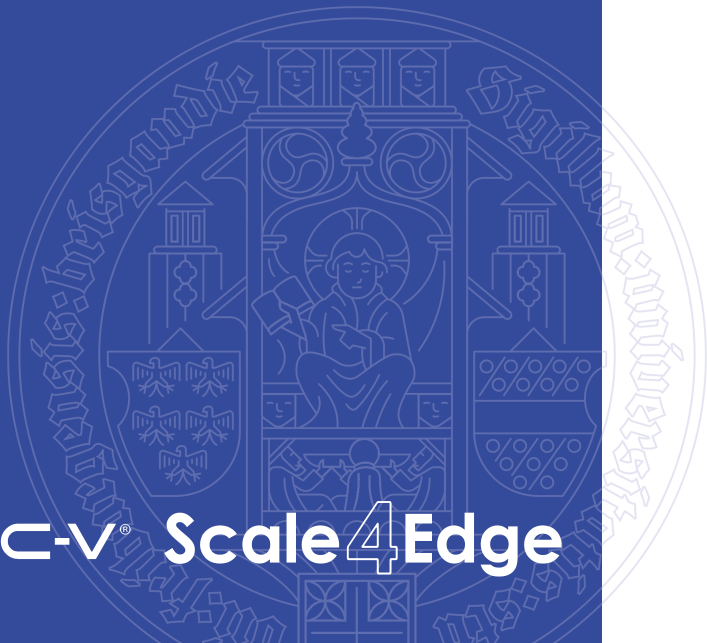
Test and Reliability

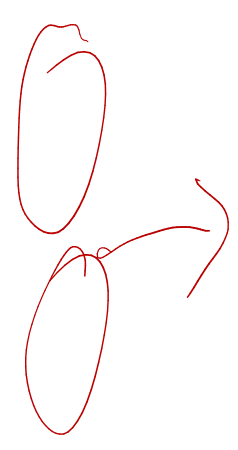
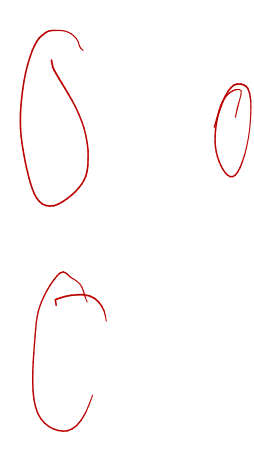
Chapter 7: Software-Based Self-Test

Tobias Faller, M.Sc.
Prof. Dr. Ralf Wimmer
Dr. Tobias Seufert

Summer Term 2026

 **RISC-V®** 

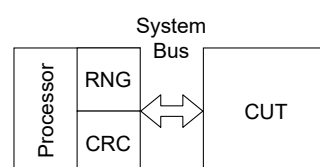
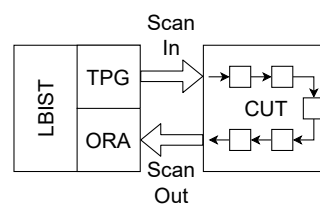


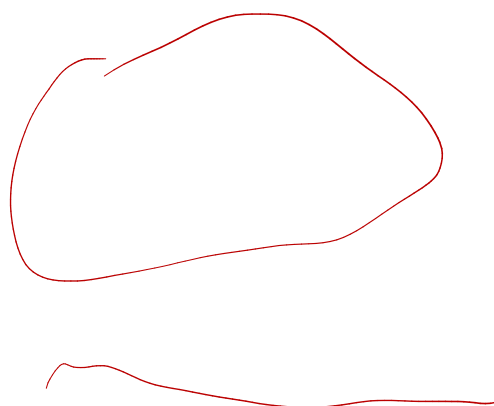


Motivation

Software-Based Self-Test

- Design-for-Testability (DFT) **has hardware costs**
 - Additional chip area for test points and routing
 - Additional design complexity, limitations during testing
 - Can decrease clock frequency on critical paths
- Circuits degrade during lifespan (see bathtub curve)
 - Testing during operation (in-the-field) is required
 - **BIST is expensive** (TPG, ORA, routing, scan-chains)
- Modern designs often include (general-purpose) processors
 - ⇒ Use existing processor to test the circuit





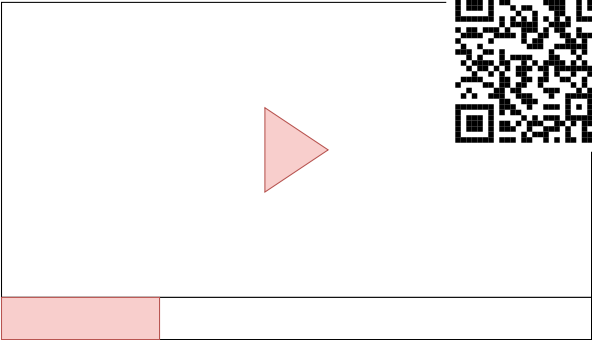
Motivation

Power-on Self-Test (POST) as Example for SBST

BIOS / UEFI POST:

```
UEFI Some Company Ltd.  
Copyright 1970-2026  
Build 06/11/26  
  
1 CPU(s) / 8 Core(s) Detected  
8 x 64 kB L1 Cache Passed  
8 x 512 kB L2 Cache Passed  
2 x 16 MB L3 Cache Passed  
4 x 8192 MB DRAM Passed  
  
Press <ESC> to BIOS Setup.
```

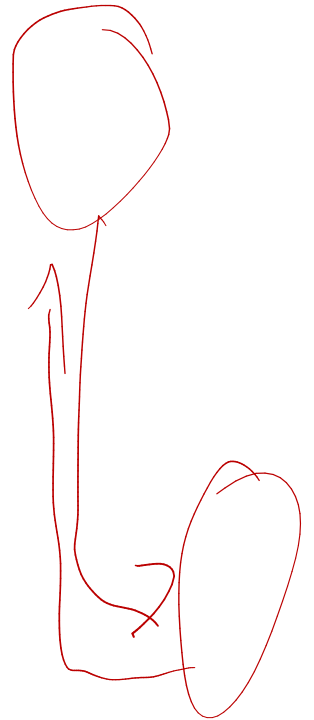
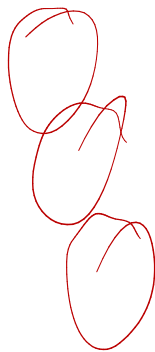
Arcade Machine POST:





Outline

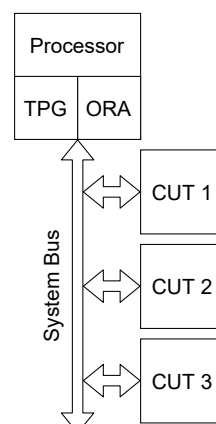
- Software-based Built-in Self-Test (SBIST)
 - Random-based Testing (Software LBIST)
Example: Software LBIST for MAC unit
 - Pattern-based Testing (Software MBIST)
Example: Software MBIST for Register File (SRAM)
Example: Software MBIST for Scratchpad Memory (DRAM)
- Constrained Automatic Test Pattern Generation (ATPG)
 - Constraint Specification via Validity Checker Module (VCM)
Example: Constrained ATPG for MAC unit
 - Test Pattern Retargeting
Example: Gemini MAC-Unit SBST (Scale4Edge)
- Software-based Self-Test (SBST)
 - Generic Constraints via Validity Checker Module (VCM)
Example: Startup SBST (Scale4Edge)
Example: Online SBST (Scale4Edge)



Software-based Built-in Self-Test (SBIST)

Random-based Testing (Software LBIST)

- Circuit under test (CUT) is tested in functional mode
 - Test runs at full speed of hardware
 - No over-testing of circuit
- Processor takes on role of external tester
 - Processor controls inputs (TPG) to circuit under test (CUT)
 - Processor observes outputs (ORA) from circuit under test (CUT)
 - Processor verifies test output (ATE)
- System bus as interconnect between processor and CUTs
 - Control inputs to circuit by using **write transactions**
 - Observe outputs from circuit by using **read transactions**



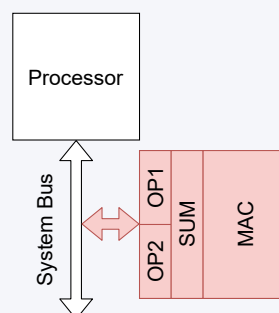
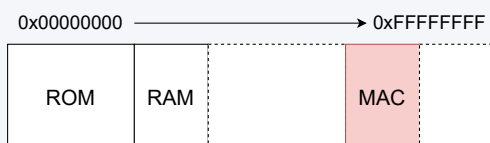
Software-based Built-in Self-Test (SBIST)

Example: Software LBIST for MAC unit

- MAC unit performs Multiply Accumulate operations (MAC)

$$\text{SUM}_{\text{Result}} = \text{SUM}_{\text{Initial}} + \sum_{i=0}^n \text{OP1}_i \cdot \text{OP2}_i$$

- MAC unit is integrated as memory mapped I/O (MMIO) device
 - Operands written via memory mapped registers (OP1, OP2)
 - Resulting sum read via memory mapped register (SUM)



LFSA

LFSA

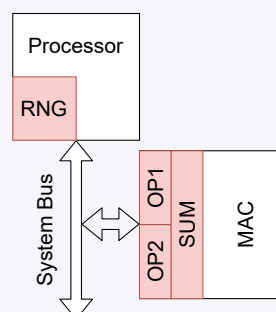
Software-based Built-in Self-Test (SBIST)

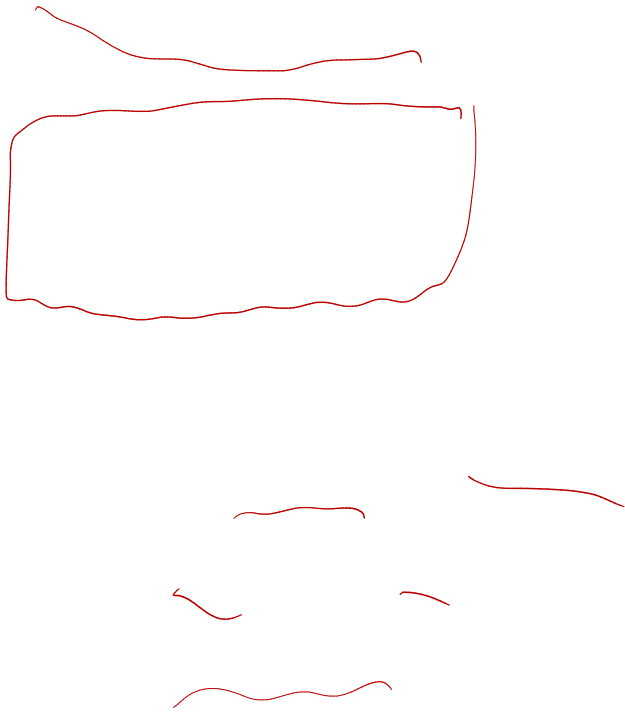
Example: Software LBIST for MAC unit

Software LBIST for memory mapped MAC unit

- Executed steps by processor core:

- TPG: Generate test inputs via **Random Number Generator (RNG)** and apply inputs to CUT ~~one after the other~~
 $\forall i \in \{0, \dots, n\} : OP1_i := \text{RNG}(2i)$
 $OP2_i := \text{RNG}(2i + 1)$
WRITE($OP1_i, OP2_i$)
- ORA: Observe test outputs from CUT
 $SUM_{cut} := \text{READ}(SUM)$
- ATE: Verify result by comparing to expected output
 $SUM_{expected} := \sum_{i=0}^n OP1_i \cdot OP2_i$

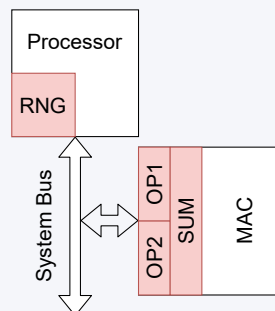




Software-based Built-in Self-Test (SBIST)

Example: Software LBIST for MAC unit

```
int sbist_mac(uint32_t seed, uint32_t multiplications) {  
    // Mersenne Twister Random Number Generator  
    std::mt19937 random(seed);  
  
    uint64_t expected = 0ull;  
    mac0.sum = expected;  
    for (uint32_t i = 0u; i < multiplications; i++) {  
        auto a = random(), b = random();  
        mac0.op1 = a; mac0.op2 = b;  
        expected += a * b;  
    }  
    return (mac0.sum != expected) ? FAULTY : OKAY;  
}
```



Handwritten red scribbles.

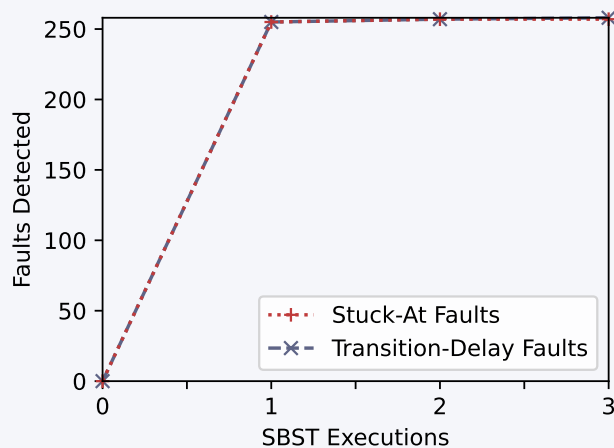
Handwritten red lines.

Large handwritten red oval.

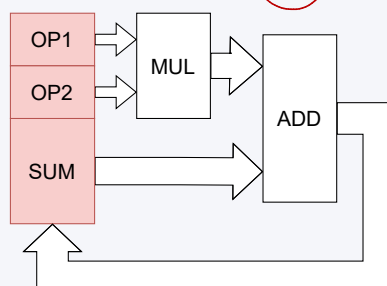
Software-based Built-in Self-Test (SBIST)

Example: Software LBIST for MAC unit

RTL faults in OP1, OP2 and SUM registers:



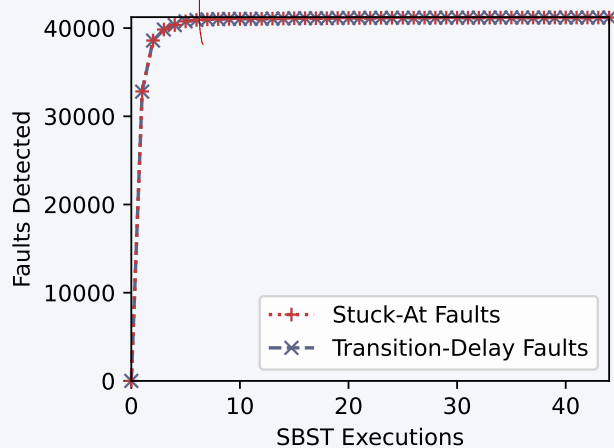
- 3 SBST executions required
- 100 % stuck-at fault coverage
- 100 % transition-delay fault coverage



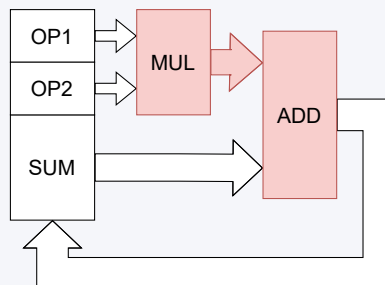
Software-based Built-in Self-Test (SBIST)

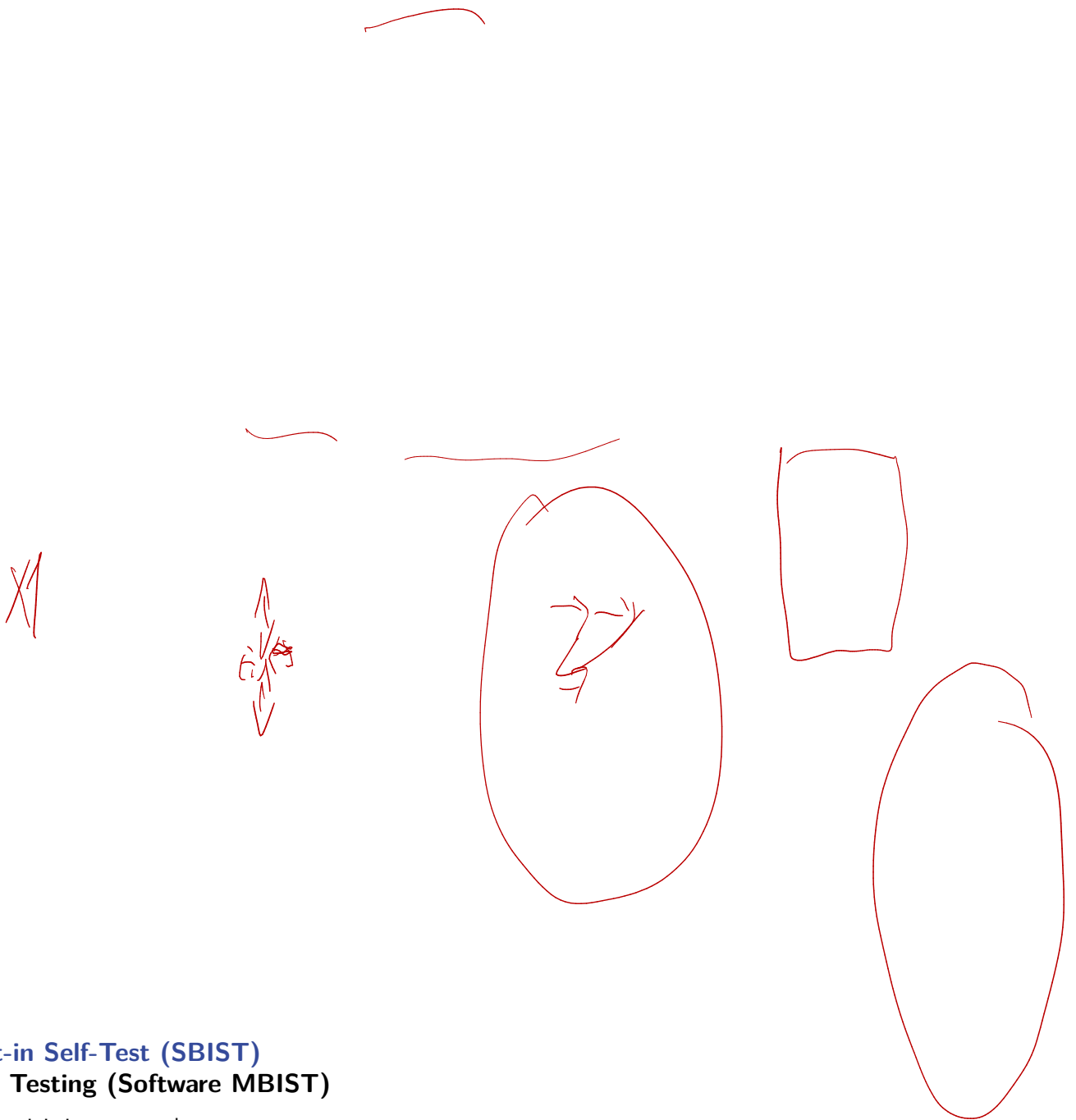
Example: Software LBIST for MAC unit

Gate-level faults in MUL and ADD circuits:



- ≥ 50 SBST executions required
- 88 faults remain untested
- 99.89 % stuck-at fault coverage
- 99.89 % transition-delay fault coverage





Software Built-in Self-Test (SBIST)

Pattern-based Testing (Software MBIST)

- Generate deterministic structured patterns
- Use regularity of circuit under test as advantage
 - Generation is algorithmically cheap and fast
 - Heavily used for memory (DRAM) and register bank (SRAM) testing

0	0	0
0	0	0
0	0	0

1	1	1
1	1	1
1	1	1

Fill Pattern

0	1	0
1	0	1
0	1	0

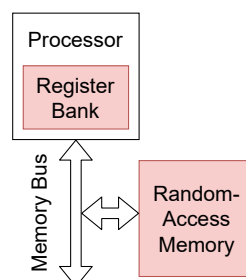
1	0	1
0	1	0
1	0	1

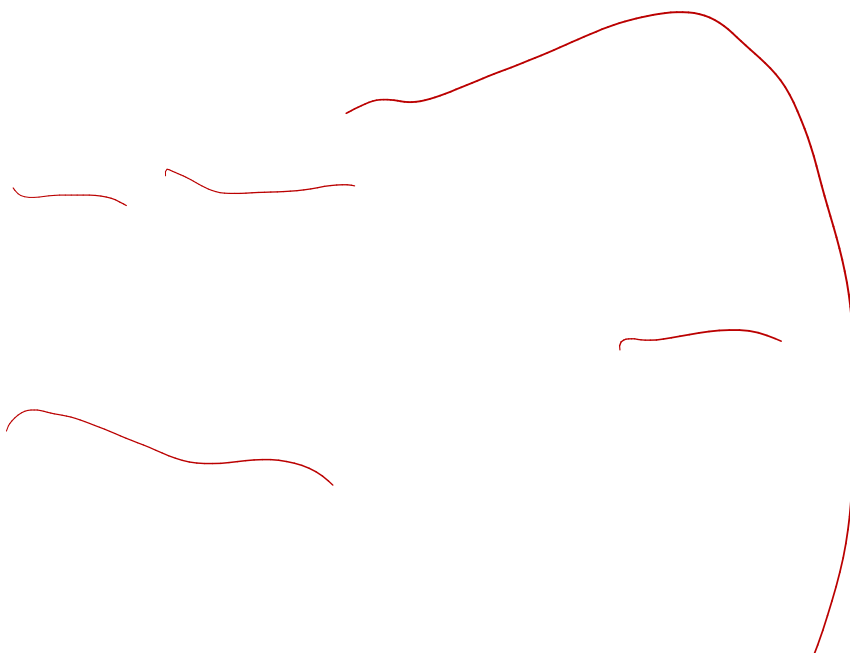
Checkerboard Pattern

0	0	0
0	0	0
0	1	1

1	1	1
1	1	0
0	0	0

Marching Pattern

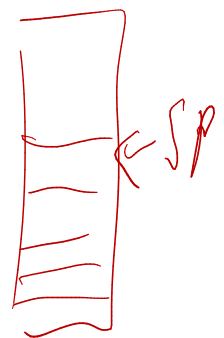
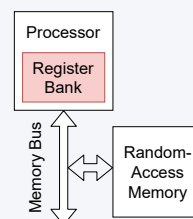




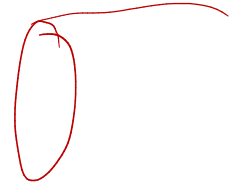
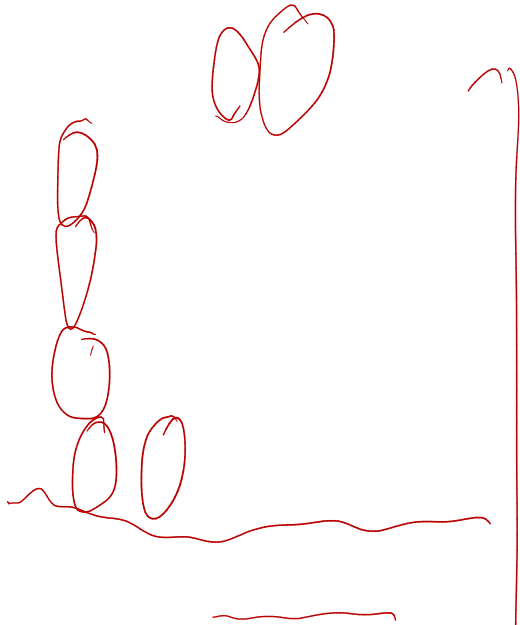
Software Built-in Self-Test (SBIST)

Example: Software MBIST for Register File (SRAM)

- Executed steps by processor core:
 - Test each register separately
 $\forall reg \in \{x1, x2, \dots, x31\}$:
 - (Save register content to be restored later)
 $TMP_{reg} := READ(reg)$
 - Write pattern to register (e.g.: $PATTERN := PFFFFFFF_{16}$)
 $WRITE(reg, PATTERN)$
 - Read and compare pattern from register
 $RESULT_{cut} := READ(reg)$
 $RESULT_{expected} := PATTERN$
 - (Restore original register content)
 $WRITE(reg, TMP_{reg})$



WD



Software Built-in Self-Test (SBIST)

Example: Software MBIST for Register File (SRAM)

- Fill pattern (ones) to detect stuck-at 0 faults in x1
- Temporary registers t0 (pattern), t1 (original x1 content)
- Testing other registers is equivalent to x1

```
sbist_regfile_x1_sa0:
    mv t1, x1 ; Save x1
    li t0, 0xFFFFFFFF
    li x1, 0xFFFFFFFF
    bne x1, t0, .reg_x1_faulty
    mv x1, t1 ; Restore x1
    li a0, 0 ; Return OKAY
    ret
.reg_x1_faulty:
    ; Handle appropriately
```

Processor							
Register Bank							
x1	1	0	1	1	1	1	...
x2							
...							
t0	1	1	1	1	1	1	...
t1	X	X	X	X	X	X	...

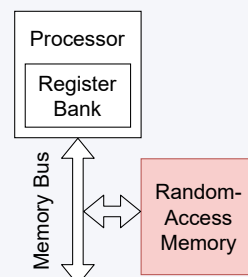
0

~

Software Built-in Self-Test (SBIST)

Example: Software MBIST for Scratchpad Memory (DRAM)

- Memory contents are fully overwritten (usually run as part of POST before firmware starts)
- Executed steps by processor core:
 1. Write pattern to memory cells (e.g.: $PATTERN := FF_{16}$)
 $\forall i \in \{0, \dots, size - 1\} :$
WRITE(*i*, ***PATTERN***)
 2. Read and compare pattern from memory cells
 $\forall i \in \{0, \dots, size - 1\} :$
RESULT_{cut} := ***READ***(*i*)
RESULT_{expected} := ***PATTERN***





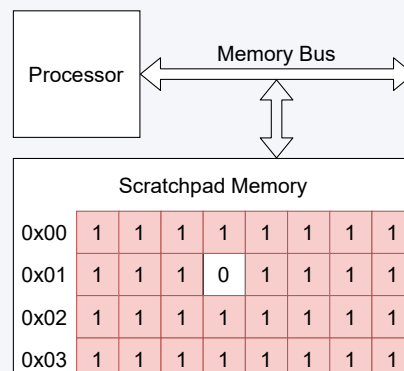
Software Built-in Self-Test (SBIST)

Example: Software MBIST for Scratchpad Memory (DRAM)

Fill pattern (e.g.: PATTERN := 00_{16} and FF_{16})

```
int sbist_scratchpad_fill(void* start, void* end) {
    for (uint8_t volatile* p = start; p != end; p++) {
        *p = 0x00u;
        if (*p != 0x00u) return FAULTY;

        *p = 0xFFu;
        if (*p != 0xFFu) return FAULTY;
    }
    return OKAY;
}
```



010101.. 101010..

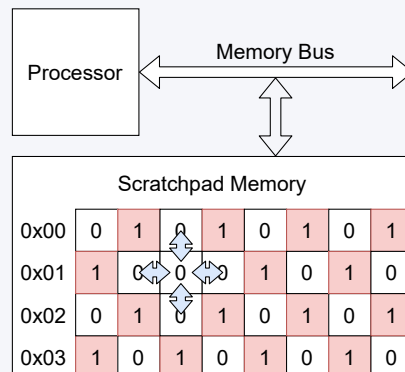
~~~~~

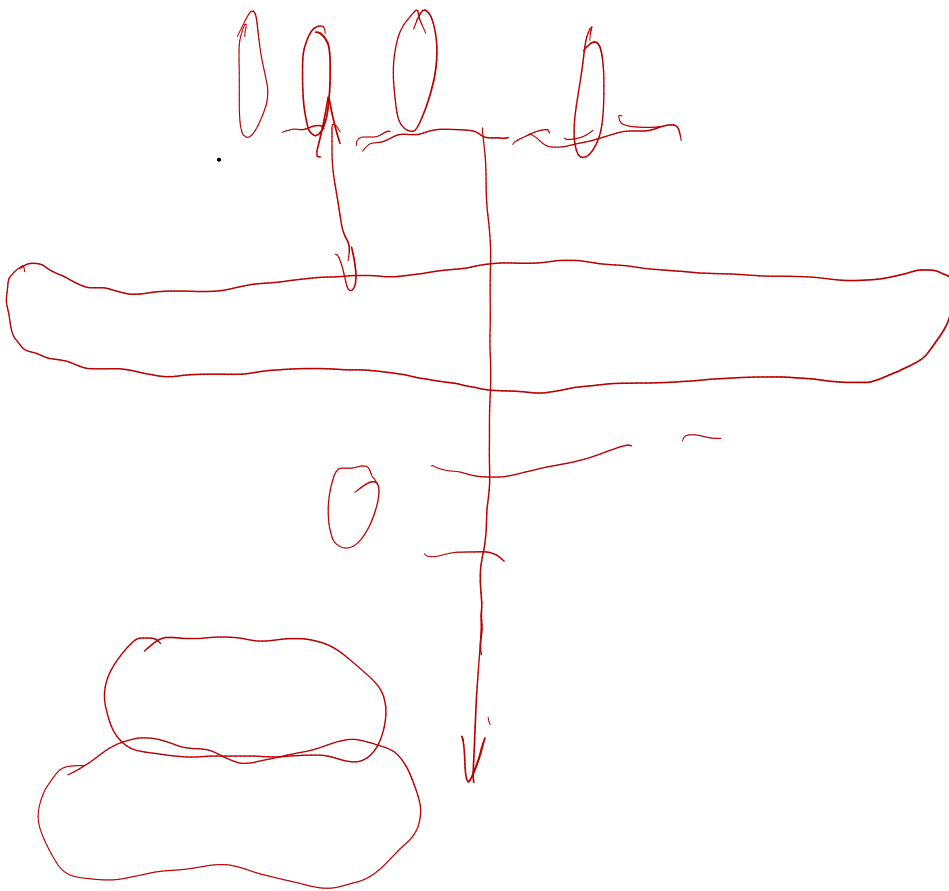
## Software Built-in Self-Test (SBIST)

### Example: Software MBIST for Scratchpad Memory (DRAM)

Checkerboard pattern (e.g.: PATTERN := 55<sub>16</sub> and AA<sub>16</sub>)

```
int sbist_scratchpad_checker(void* start, void* end) {  
    for (uint8_t volatile* p = start; p != end; p++)  
        *p = (is_row_even(p) ? 0x55u : 0xAAu);  
  
    for (uint8_t volatile* p = start; p != end; p++)  
        if (*p != (is_row_even(p) ? 0x55u : 0xAAu))  
            return FAULTY;  
  
    return OKAY;  
}
```



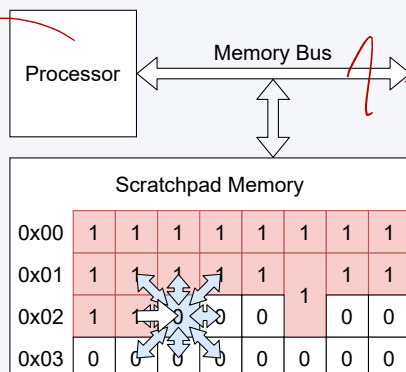


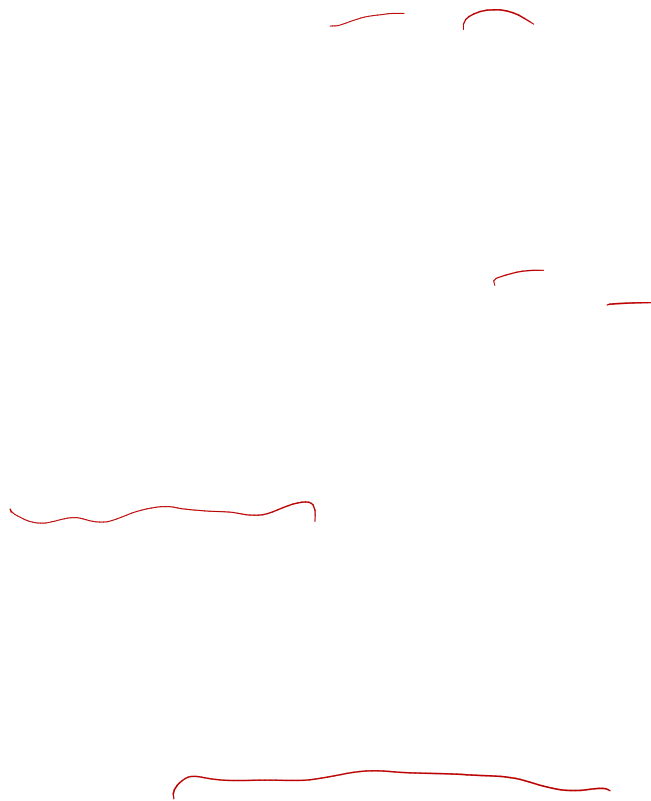
## Software Built-in Self-Test (SBIST)

### Example: Software MBIST for Scratchpad Memory (DRAM)

March pattern { $\uparrow(w0)$ ;  $\uparrow(r0, w1)$ ;  $\downarrow(r1, w0)$ }

```
int sbist_scratchpad_march(void* start, void* end) {
    std::memset(start, 0u, (uint8_t*)end - (uint8_t*)start);
    for (size_t i = 0u; i < (end - start) * 8u; i++) {
        uint8_t volatile* p = start + (i / 8u);
        uint8_t pattern = (0xFFu >> (8u - (i % 8u)));
        if (*p != pattern) return FAULTY;
        *p |= (pattern << 1u) | 1u; // Set next bit to 1
    }
    for (size_t i = 0u; i < (end - start) * 8u; i++) {
        uint8_t volatile* p = end - (1u + i / 8u);
        uint8_t pattern = (0xFFu >> (i % 8u));
        if (*p != pattern) return FAULTY;
        *p &= (pattern >> 1u); // Set next bit to 0
    }
    return OKAY;
}
```

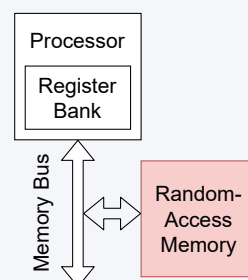




## Software Built-in Self-Test (SBIST)

### Example: Software MBIST for Scratchpad Memory (DRAM)

1. Fill pattern ( $\text{PATTERN} := 00_{16}$  and  $\text{FF}_{16}$ )
  - ✓ Stuck-at 0 / 1 faults
2. Checkerboard pattern ( $\text{PATTERN} := 55_{16}$  and  $\text{AA}_{16}$ )
  - ✓ Stuck-at 0 / 1 faults
  - ✓ Horizontal bridge faults
  - ✓ Vertical bridge faults
3. March pattern  $\{\uparrow\downarrow (w0); \uparrow (r0, w1); \downarrow\uparrow (r1, w0)\}$ 
  - ✓ Stuck-at 0 / 1 faults
  - ✓ Horizontal bridge faults
  - ✓ Vertical bridge faults
  - ✓ Diagonal bridge faults



## Software Built-in Self-Test (SBIST)

### Pattern-based Testing (Software MBIST)

Different march elements test for different faults (table for reference only)

| Fault                 | Behavior                             | Elements                                                                   |
|-----------------------|--------------------------------------|----------------------------------------------------------------------------|
| Stuck-at (SAF)        | Stuck-at 0 ( $\Rightarrow 0$ )       | $\Downarrow (w1); \Downarrow (r1)$                                         |
|                       | Stuck-at 1 ( $\Rightarrow 1$ )       | $\Downarrow (w0); \Downarrow (r0)$                                         |
| Transition-delay (TF) | Slow-to-rise ( $0w1 \Rightarrow 0$ ) | $\Downarrow (w0); \Downarrow (w1, r1)$                                     |
|                       | Slow-to-fall ( $1w0 \Rightarrow 1$ ) | $\Downarrow (w1); \Downarrow (w0, r0)$                                     |
| State (SF)            | randomly $\Rightarrow 0$             | $\Downarrow (w1); \Downarrow (r1)$                                         |
|                       | randomly $\Rightarrow 1$             | $\Downarrow (w0); \Downarrow (r0)$                                         |
| Address decoder (ADF) | w0 $\Rightarrow$ other 0             | $\Downarrow (w1); \Uparrow (r1, w0); \Downarrow (w1); \Downarrow (r1, w0)$ |
|                       | w1 $\Rightarrow$ other 1             | $\Downarrow (w0); \Uparrow (r0, w1); \Downarrow (w0); \Downarrow (r0, w1)$ |

## Software Built-in Self-Test (SBIST)

### Pattern-based Testing (Software MBIST)

Different march elements test for different faults (table for reference only)

| Fault                             | Behavior            | Elements                               |
|-----------------------------------|---------------------|----------------------------------------|
| Incorrect read (IRF)              | 0r1 $\Rightarrow$ 0 | $\Downarrow (w0); \Downarrow (r0)$     |
|                                   | 1r0 $\Rightarrow$ 1 | $\Downarrow (w1); \Downarrow (r1)$     |
| Deceptive read destructive (DRDF) | 0r0 $\Rightarrow$ 1 | $\Downarrow (w0); \Downarrow (r0, r0)$ |
|                                   | 1r1 $\Rightarrow$ 0 | $\Downarrow (w1); \Downarrow (r1, r1)$ |
| Read disturb (RDF)                | 0r0 $\Rightarrow$ 1 | $\Downarrow (w0); \Downarrow (r0, r0)$ |
|                                   | 1r1 $\Rightarrow$ 0 | $\Downarrow (w1); \Downarrow (r1, r1)$ |
| Write disturb (WDF)               | 0w0 $\Rightarrow$ 1 | $\Downarrow (w0); \Downarrow (w0, r0)$ |
|                                   | 1w1 $\Rightarrow$ 0 | $\Downarrow (w1); \Downarrow (w1, r1)$ |

## Software Built-in Self-Test (SBIST)

### Pattern-based Testing (Software MBIST)

Different march elements test for different faults (table for reference only)

| Fault                        | Behavior        | Elements               |
|------------------------------|-----------------|------------------------|
| Disturb coupling (CFds rx)   | r0 ⇒ coupled 0  | ↕ (w1); ↘ (r1, w0, r0) |
|                              | r0 ⇒ coupled 1  | ↕ (w0); ↘ (r0)         |
|                              | r1 ⇒ coupled 0  | ↕ (w1); ↘ (r1)         |
|                              | r1 ⇒ coupled 1  | ↕ (w0); ↘ (r0, w1, r1) |
| Disturb coupling (CFds xwx)  | 0w0 ⇒ coupled 0 | ↕ (w1); ↘ (r1, w0, w0) |
|                              | 0w0 ⇒ coupled 1 | ↕ (w0); ↘ (r0, w0)     |
|                              | 1w1 ⇒ coupled 0 | ↕ (w1); ↘ (r1, w1)     |
|                              | 1w1 ⇒ coupled 1 | ↕ (w0); ↘ (r0, w1, w1) |
| Disturb coupling (CFds xw!x) | 0w1 ⇒ coupled 0 | ↕ (w1); ↘ (r1, w0, w1) |
|                              | 0w1 ⇒ coupled 1 | ↕ (w0); ↘ (r0, w1)     |
|                              | 1w0 ⇒ coupled 0 | ↕ (w1); ↘ (r1, w0)     |
|                              | 1w0 ⇒ coupled 1 | ↕ (w0); ↘ (r0, w1, w0) |

## Software Built-in Self-Test (SBIST)

### Pattern-based Testing (Software MBIST)

Different march elements test for different faults (table for reference only)

| Fault                          | Behavior                               | Elements                                   |
|--------------------------------|----------------------------------------|--------------------------------------------|
| State / static coupling (CFst) | 0 $\Rightarrow$ coupled 0              | $\Downarrow (w1); \Downarrow (r1, w0)$     |
|                                | 0 $\Rightarrow$ coupled 1              | $\Downarrow (w0); \Downarrow (r0)$         |
|                                | 1 $\Rightarrow$ coupled 0              | $\Downarrow (w1); \Downarrow (r1)$         |
|                                | 1 $\Rightarrow$ coupled 1              | $\Downarrow (w0); \Downarrow (r0, w1)$     |
| Inversion coupling (CFin)      | 0w1, coupled 0 $\Rightarrow$ coupled 1 | $\Downarrow (w0); \Downarrow (r0, w1)$     |
|                                | 0w1, coupled 1 $\Rightarrow$ coupled 0 | $\Downarrow (w1); \Downarrow (r1, w0, w1)$ |
|                                | 1w0, coupled 0 $\Rightarrow$ coupled 1 | $\Downarrow (w0); \Downarrow (r0, w1, w0)$ |
|                                | 1w0, coupled 1 $\Rightarrow$ coupled 0 | $\Downarrow (w1); \Downarrow (r1, w0)$     |
| Idempotent coupling (CFid)     | 0w1 $\Rightarrow$ coupled 0            | $\Downarrow (w1); \Downarrow (r1, w0, w1)$ |
|                                | 0w1 $\Rightarrow$ coupled 1            | $\Downarrow (w0); \Downarrow (r0, w1)$     |
|                                | 1w0 $\Rightarrow$ coupled 0            | $\Downarrow (w1); \Downarrow (r1, w0)$     |
|                                | 1w0 $\Rightarrow$ coupled 1            | $\Downarrow (w0); \Downarrow (r0, w1, w0)$ |

... and many many more

## Software Built-in Self-Test (SBIST)

### Pattern-based Testing (Software MBIST)

| Algorithm | $\Theta(N)$ | Elements                                                                                                                                                                          |
|-----------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MATS+     | $5N$        | $\{\updownarrow (w0); \uparrow (r0, w1); \downarrow (r1, w0)\}$                                                                                                                   |
| MATS++    | $6N$        | $\{\uparrow (w0); \uparrow (r0, w1); \downarrow (r1, w0, r0)\}$                                                                                                                   |
| March C-  | $10N$       | $\{\uparrow (w0); \uparrow (r0, w1); \uparrow (r1, w0); \downarrow (r0, w1); \downarrow (r1, w0); \updownarrow (r0)\}$                                                            |
| March C+  | $14N$       | $\{\uparrow (w0); \uparrow (r0, w1, r1); \uparrow (r1, w0, r0); \downarrow (r0, w1, r1); \downarrow (r1, w0, r0); \updownarrow (r0)\}$                                            |
| March A   | $15N$       | $\{\uparrow (w0); \uparrow (r0, w1, w0, w1); \uparrow (r1, w0, w1);$<br>$\downarrow (r1, w0, w1, w0); \downarrow (r0, w1, w0)\}$                                                  |
| March B   | $17N$       | $\{\uparrow (w0); \uparrow (r0, w1, r1, w0, r0, w1); \uparrow (r1, w0, w1);$<br>$\downarrow (r1, w0, w1, w0); \downarrow (r0, w1, w0)\}$                                          |
| March SR  | $14N$       | $\{\downarrow (w0); \uparrow (r0, w1, r1, w0); \downarrow (r0, r0); \uparrow (w1); \downarrow (r1, w0, r0, w1); \uparrow (r1, r1); \}$                                            |
| March SS  | $22N$       | $\{\updownarrow (w0); \uparrow (r0, r0, w0, r0, w1); \uparrow (r1, r1, w1, r1, w0);$<br>$\downarrow (r0, r0, w0, r0, w1); \downarrow (r1, r1, w1, r1, w0); \updownarrow (r0); \}$ |
| March X   | $6N$        | $\{\uparrow (w0); \uparrow (r0, w1); \downarrow (r1, w0); \updownarrow (r0)\}$                                                                                                    |
| March Y   | $22N$       | $\{\updownarrow (w0); \uparrow (w0, r0, w1, w1, r1); \uparrow (w1, r1, w0, w0, r0);$<br>$\downarrow (r0, w0, w1, w1, r1); \downarrow (r1, w1, w0, w0, r0); \updownarrow (r0)\}$   |

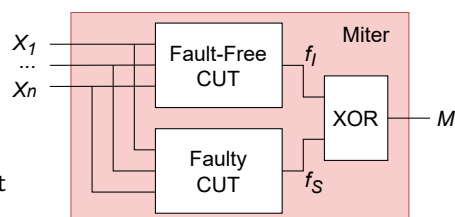
## Software Built-in Self-Test (SBIST) - References I

- [1] M. Masnita et al. "March-based SRAM diagnostic algorithm for distinguishing Stuck-At and transition faults". In: *IEICE Electronics Express* 6 (Aug. 2009), pp. 1091–1097. DOI: [10.1587/elex.6.1091](https://doi.org/10.1587/elex.6.1091).
- [2] Yongwen Wang, Qianbing Zheng, and Yin Yuan. "The Improvement of March C+ Algorithm for Embedded Memory Test". In: *Computer Engineering and Technology*. Ed. by Weixia Xu et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016, pp. 31–37. ISBN: 978-3-662-49283-3.
- [3] Abhilash Kaushal. *Memory fault models and testing*. June 2015. URL: <https://www.edn.com/memory-fault-models-and-testing/> (visited on 09/07/2026).

## Constrained Automatic Test Pattern Generation (ATPG)

### Reprise SAT-based Automatic Test Pattern Generation (ATPG)

- Miter circuit of fault-free  $f_I$  and faulty  $f_S$  circuits
- Miter output  $M := f_I \oplus f_S$  shows the detection of fault
  - Equivalent inputs  $(X_1, \dots, X_n)$  for both circuits
  - Different output of circuits ( $f_I$  vs  $f_S$ )
  - ⇒ Pattern  $P := (X_1, \dots, X_n)$  can differentiate circuits
- For multiple outputs any of the outputs can detect the fault



$$M := \bigvee_{i \in \{1, \dots, N_{\text{Out}}\}} (f_{I_i} \oplus f_{S_i})$$

## Constrained Automatic Test Pattern Generation (ATPG)

### Reprise Iterative Logic Array Model (Sequential Unrolling)

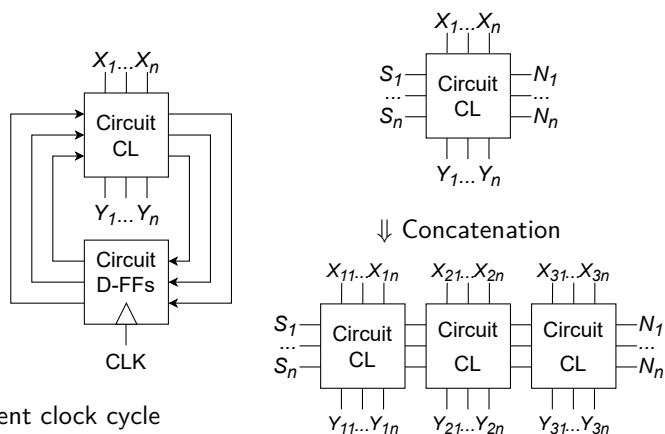
- Sequential circuits have two parts ...

- Combinational logic (CL)
- Sequential D-Flip-Flops (D-FFs)

- A timeframe consists of ...

- Combinational logic (CL)
- Inputs  $X := (X_1, \dots, X_n)$
- State  $S := (S_1, \dots, S_n)$
- Output  $Y := \text{CL}_{\text{Out}}(X, S)$
- Next State  $N := \text{CL}_{\text{Next}}(X, S)$

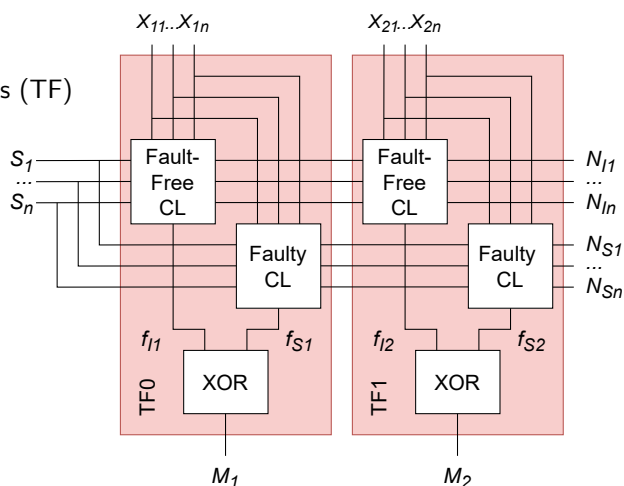
- Unrolling circuit over timeframes, each representing circuit during a different clock cycle



## Constrained Automatic Test Pattern Generation (ATPG)

### Reprise Bounded Model Checking (BMC)-based ATPG

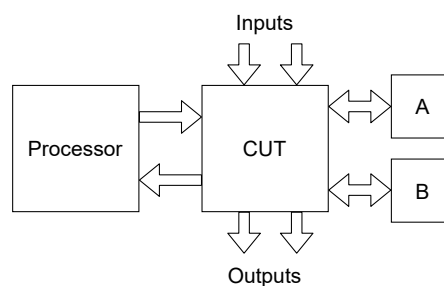
- Circuit is iteratively unrolled for  $k$  timeframes (TF)
- Check if fault can be tested in timeframe  $k$ 
  - If  $M_k = 1$ , then fault testable in  $k$  TFs and pattern  $P := (S, X_1, \dots, X_n)$  found
  - If  $M_k = 0$ , then continue with  $k := k + 1$
- Usually starting state is constrained to be all zeroes  $S := (0, \dots, 0)$  like after reset or all unknowns  $S := (X, \dots, X)$



## Constrained Automatic Test Pattern Generation (ATPG)

### Constraint specification via Validity Checker Module (VCM)

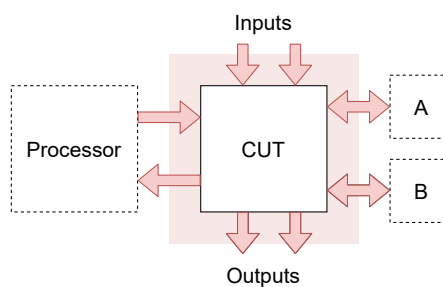
- Circuit under test (CUT) is embedded in system
  - System has availability requirements (real-time)
  - System is in operating mode (in-the-field)
- Testing CUT has additional complexity ...
  - Not all inputs can be controlled
  - Not all outputs can be observed
  - The state is controlled indirectly
  - The state is observed indirectly



## Constrained Automatic Test Pattern Generation (ATPG)

### Constraint specification via Validity Checker Module (VCM)

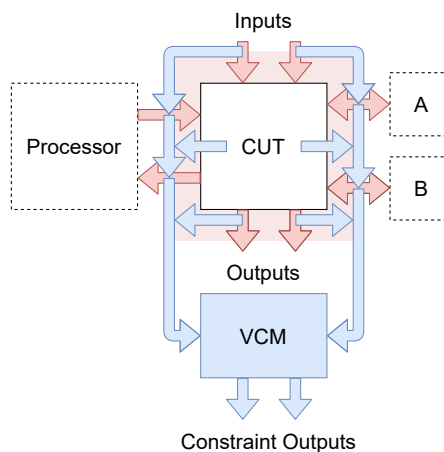
- Use Automatic Test Pattern Generation (ATPG)
  - Generate test patterns for CUT as in system
  - Later transform test patterns into SBST
- Circuit under test (CUT) is embedded in system
  - Replace system parts with model for test generation
  - System model can be specified as test constraints
  - Environment can be specified as test constraints
- System and environment are specified as constraints
  - CUT inputs / outputs
  - CUT bus protocol with processor
  - CUT interactions with system parts



## Constrained Automatic Test Pattern Generation (ATPG)

### Constraint specification via Validity Checker Module (VCM)

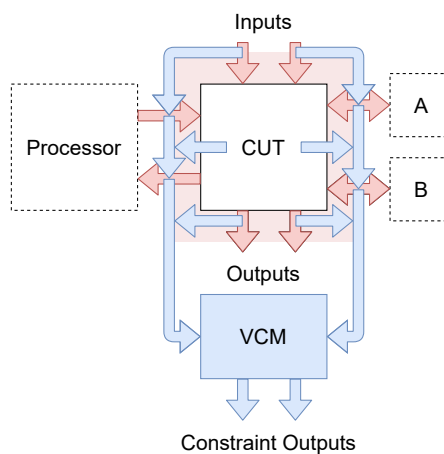
- Validity Checker Module (VCM) [1], [2] observes the CUT
  - CUT inputs / outputs
  - CUT bus protocol with processor
  - CUT interactions with system parts
  - CUT internal state if required
- VCM evaluates constraints for CUT
  - Evaluation result is provided via constraint outputs
  - If constraint is falsified, then the output is 0
  - If constraint is satisfied, then the output is 1
- Constraint outputs are forced to 1 during ATPG
  - ⇒ The constraints are propagated to the VCM inputs
  - ⇒ The constraints are applied to the CUT



## Constrained Automatic Test Pattern Generation (ATPG)

### Constraint specification via Validity Checker Module (VCM)

- VCM observes the CUT
  - CUT inputs / outputs
  - CUT bus protocol with processor
  - CUT interactions with system parts
  - CUT internal state if required
- VCM evaluates constraints for CUT
  - Evaluation result is provided via constraint outputs
  - If constraint is falsified, then the output is 0
  - If constraint is satisfied, then the output is 1

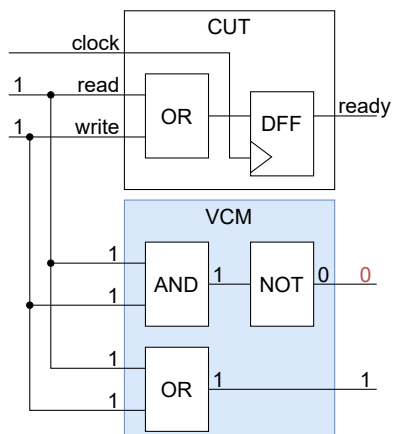


## Constrained Automatic Test Pattern Generation

### Constraint specification via Validity Checker Module (VCM)

- VCM evaluates constraints for CUT
  - Evaluation result is provided via constraint outputs
  - If constraint is falsified, then the output is 0
  - If constraint is satisfied, then the output is 1

```
module vcm(  
    input read, input write,  
    output constraint_not_both,  
    output constraint_transaction  
);  
    assign constraint_not_both = !(read & write);  
    assign constraint_transaction = read | write;  
endmodule
```



## Constrained Automatic Test Pattern Generation

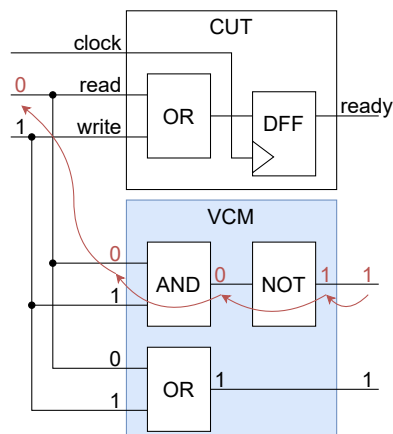
### Constraint specification via Validity Checker Module (VCM)

- VCM evaluates constraints for CUT
  - Evaluation result is provided via constraint outputs
  - If constraint is falsified, then the output is 0
  - If constraint is satisfied, then the output is 1
- Constraint outputs are forced to 1 during ATPG
  - ⇒ The constraints are propagated to the VCM inputs
  - ⇒ The constraints are applied to the CUT

```

module vcm(
    input read, input write,
    output constraint_not_both,
    output constraint_transaction
);
    assign constraint_not_both = !(read & write);
    assign constraint_transaction = read | write;
endmodule

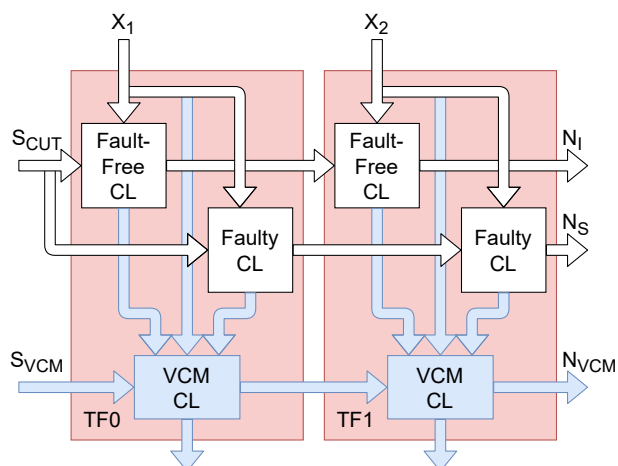
```



## Constrained Automatic Test Pattern Generation

### Constraint specification via Validity Checker Module (VCM)

- VCM is unrolled together with miter circuit
  - Each timeframe is equivalent to a constrained combinational ATPG instance
  - Constraints are applied independently each timeframe
- VCM can contain sequential state
  - Unrolling of timeframes like CUT (iterative logic array model)
  - Required for stateful constraints
    - bus protocols
    - state machines
    - ...



## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Constrained ATPG for MAC unit

- Environment constraints

- reset: always inactive (system running)

- System constraints

- read / write: never active at the same time

- address: always in range during transactions

- write\_data: no constraints required

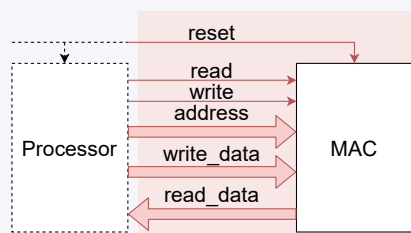
- read\_data: no constraints, controlled by MAC unit

- ATPG constraints

- all inputs: equivalent for fault-free / faulty CUT

- Fault propagation constraints

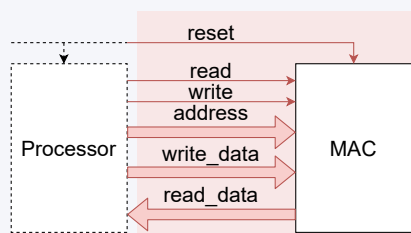
- read / read\_data: fault propagated via data during read



## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Constrained ATPG for MAC unit

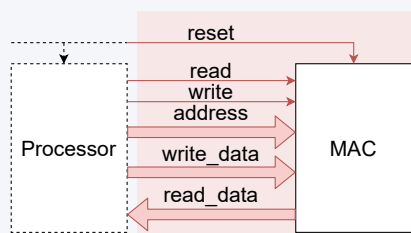
```
module mac_vcm(  
    input reset, input read, input write, input [31:0] address,  
    output environment_constraints,  
    output system_constraints  
);  
    // always inactive (system running)  
    assign environment_constraints = !reset;  
    assign system_constraints = &{  
        // never active at the same time  
        !(read && write),  
  
        // always in range and aligned transactions  
        address >= 32'h 02000000 && address <= 32'h 0200000F,  
        address[1:0] == 2'b 00  
    };  
endmodule
```



## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Constrained ATPG for MAC unit

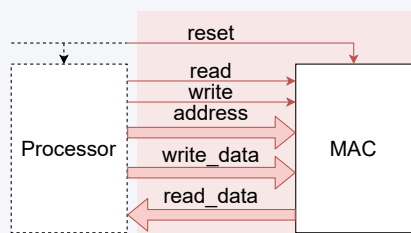
```
module mac_vcm(  
  input reset_faultfree, reset_faulty,  
  input read_faultfree, read_faulty,  
  input write_faultfree, write_faulty,  
  input [31:0] address_faultfree, address_faulty,  
  input [31:0] write_data_faultfree, write_data_faulty  
  output atpg_constraints  
);  
  assign atpg_constraints = &{  
    reset_faultfree == reset_faulty,  
    read_faultfree == read_faulty,  
    write_faultfree == write_faulty,  
    address_faultfree == address_faulty,  
    write_data_faultfree == write_data_faulty  
  };  
endmodule
```



## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Constrained ATPG for MAC unit

```
module mac_vcm(  
    input read_faultfree,  
    input read_faulty,  
    input [31:0] read_data_faultfree,  
    input [31:0] read_data_faulty,  
    output propagation_constraints  
);  
    // fault propagated via data during read  
    assign propagation_constraints = &{  
        read_faultfree,  
        read_faulty,  
        |{read_data_faultfree ^ read_data_faulty}  
    };  
endmodule
```



## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Constrained ATPG for MAC unit

| Timeframe               | 0                   | 1                   | 2                    | 3                    |
|-------------------------|---------------------|---------------------|----------------------|----------------------|
| reset                   | 0                   | 0                   | 0                    | 0                    |
| address                 | 0x02000008<br>(OP1) | 0x0200000C<br>(OP2) | 0x02000000<br>(SUML) | 0x02000000<br>(SUML) |
| read                    | 0                   | 0                   | 0                    | 1                    |
| read_data               | 0x00000000          | 0x00000000          | 0x00000000           | 0xA7485BD2           |
| read_data (Faulty)      | 0x00000000          | 0x00000000          | 0x00000000           | 0xA9485BD2           |
| write                   | 1                   | 1                   | 0                    | 0                    |
| write_data              | 0x00D97F5A          | 0xCEFD FEAD         | 0x00000000           | 0x00000000           |
| environment_constraints | 1                   | 1                   | 1                    | 1                    |
| system_constraints      | 1                   | 1                   | 1                    | 1                    |
| atpg_constraints        | 1                   | 1                   | 1                    | 1                    |
| propagation_constraints | 0                   | 0                   | 0                    | 1                    |

Constrained Automatic Test Pattern Generation (ATPG)

Example: Constrained ATPG for MAC unit

| Timeframe          | 0                   | 1                   | 2     | 3                    |
|--------------------|---------------------|---------------------|-------|----------------------|
| address            | 0x02000008<br>(OP1) | 0x0200000C<br>(OP2) | ----- | 0x02000000<br>(SUML) |
| read               | -                   | -                   | -     | 1                    |
| read_data          | -----               | -----               | ----- | 0xA7485BD2           |
| read_data (Faulty) | -----               | -----               | ----- | 0xA9485BD2           |
| write              | 1                   | 1                   | -     | -                    |
| write_data         | 0x00D97F5A          | 0xCEFD FEAD         | ----- | -----                |

```
int sbist_mac_test() {
    mac0.op1 = 0x00D97F5A;
    mac0.op2 = 0xCEFD FEAD;
    return (mac0.suml != 0xA7485BD2) ? FAULTY : OKAY;
}
```

## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Constrained ATPG for MAC unit

|                                  | Fault Coverage | Fault Efficiency | Runtime   | Transactions |
|----------------------------------|----------------|------------------|-----------|--------------|
| Full-Scan Stuck-At ATPG          | 98.27 %        | 100.00 %         | 0.20 min  | ---          |
| Sequential Stuck-At ATPG         | 98.27 %        | 100.00 %         | 3.12 min  | ---          |
| Functional Stuck-At ATPG         | 97.03 %        | 99.65 %          | 12.38 min | 1,955        |
| Full-Scan Transition-Delay ATPG  | 93.97 %        | 99.88 %          | 24.05 min | ---          |
| Sequential Transition-Delay ATPG | 93.97 %        | 99.99 %          | 29.83 min | ---          |
| Functional Transition-Delay ATPG | 92.76 %        | 99.53 %          | 22.97 min | 3,123        |

## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Constrained ATPG for MAC unit

Runtime SBST of  $N_{\text{Ins}}$  instructions, at  $F_{\text{CLK}} = 1 \text{ MHz}$ , and Instructions Per Cycle  $\text{IPC} = 0.75 \text{ Ins}/1$

$$T_{\text{SBST}} \approx \frac{N_{\text{Ins}}}{F_{\text{CLK}} \cdot \text{IPC}}$$

$$\left. \begin{array}{l} T_{\text{SAF}} \approx 9,405 \text{ Ins} \div 0.75 \text{ Ins}/1 \div 1 \text{ MHz} = 12.54 \text{ ms} \\ T_{\text{TDF}} \approx 14,031 \text{ Ins} \div 0.75 \text{ Ins}/1 \div 1 \text{ MHz} = 18.71 \text{ ms} \end{array} \right\} \leq 500 \mu\text{s} \text{ desired for real-time system}$$

|                       | Fault Coverage | Transactions | Instructions | Runtime  |
|-----------------------|----------------|--------------|--------------|----------|
| Stuck-At SBST         | 97.03 %        | 1,955        | 9,405        | 12.54 ms |
| Transition-Delay SBST | 92.76 %        | 3,123        | 14,031       | 18.71 ms |

## Constrained Automatic Test Pattern Generation (ATPG)

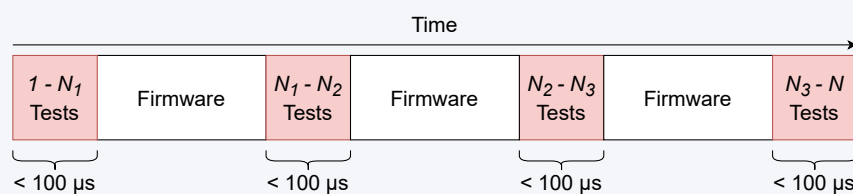
### Example: Constrained ATPG for MAC unit

- SBST's length breaks real-time property of system
- Split SBST into chunks of limited time (e.g., 100  $\mu$ s)

⇒ Periodically interleave testing with firmware execution

Instructions for time  $T_{\text{Chunk}} = 100 \mu\text{s}$ , at  $F_{\text{CLK}} = 1 \text{ MHz}$ , and Instructions Per Cycle  $\text{IPC} = 0.75 \text{ Ins}/1$

$$N_{\text{InsChunk}} \approx T_{\text{Chunk}} \cdot F_{\text{CLK}} \cdot \text{IPC} = 100 \mu\text{s} \cdot 1 \text{ MHz} \cdot 0.75 \text{ Ins}/1 = 75 \text{ Ins (quite short)}$$

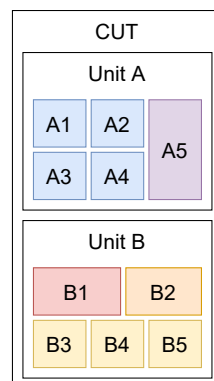


## Constrained Automatic Test Pattern Generation (ATPG) Test Pattern Retargeting

- Functional pattern generation for whole CUT is prohibitively expensive
- Modern circuits are designed using hierarchies

⇒ Use hierarchy to advantage for test creation

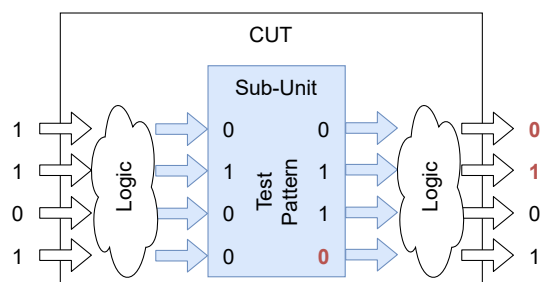
1. Generate test patterns for unique low-level units A1, A5, B1, B2, and B3
2. Retarget test patterns to equivalent units
  - Retarget test patterns from A1 to A2 to A4
  - Retarget test patterns from B3 to B4 and B5
3. Retarget test patterns up the hierarchy
  - Retarget test patterns to higher-level units A and B
  - Retarget test patterns to top-level (CUT)
4. Convert top-level (CUT) inputs into SBST



## Constrained Automatic Test Pattern Generation (ATPG) Test Pattern Retargeting

Goal: Retargeting pattern from sub-circuit to CUT

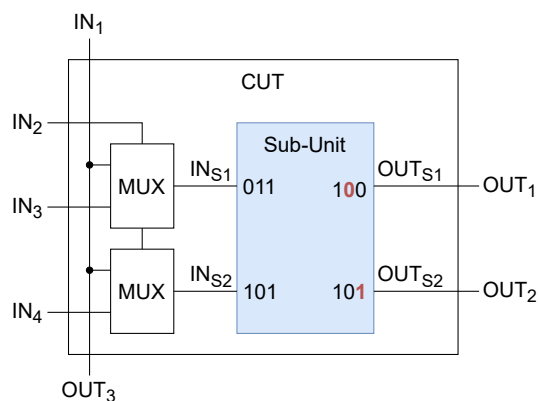
- Find inputs that satisfy inputs at sub unit
- Find inputs that propagate fault from sub unit
- The retargeted pattern can be longer
- Non-trivial problem similar to ATPG itself
  - Input assignment might not be satisfiable
  - Fault propagation might not be satisfiable
  - Both together might not be satisfiable



## Constrained Automatic Test Pattern Generation (ATPG) Test Pattern Retargeting

Strategy 1: use clever reasoning and intuition

- Assuming  $IN_2 = 0\dots$  results in  
 $IN_{S1} = 0\dots \Rightarrow IN_1 = 0\dots$  ✓  
 $IN_{S2} = 1\dots \Rightarrow IN_1 = 1\dots$  ✗
- Assuming  $IN_2 = 111$  results in  
 $IN_{S1} = 011 \Rightarrow IN_3 = 011$  ✓  
 $IN_{S2} = 101 \Rightarrow IN_4 = 101$  ✓
- Outputs always propagate  
 $OUT_{S1} = 100 \Rightarrow OUT_1 = 100$  ✓  
 $OUT_{S2} = 101 \Rightarrow OUT_2 = 101$  ✓

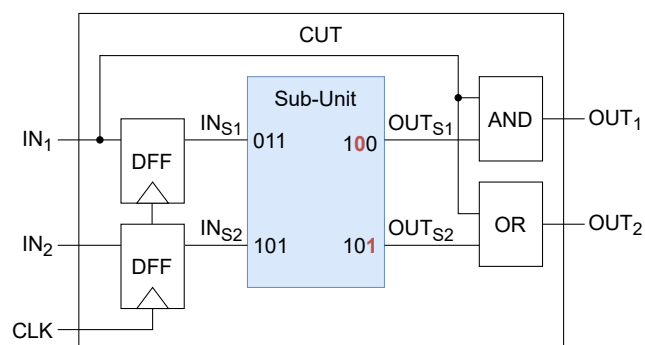


## Constrained Automatic Test Pattern Generation (ATPG)

### Test Pattern Retargeting

Strategy 2: use BMC (sub-unit as blackbox)

- Satisfying inputs for sub-circuit
  - $IN_{S1} = 011 \Rightarrow IN_1 = 011X$  ✓
  - $IN_{S2} = 101 \Rightarrow IN_2 = 101X$  ✓
  - $\Rightarrow OUT_{S1} = X100$
  - $\Rightarrow OUT_{S2} = X101$
- Propagating fault from sub-circuit
  - $OUT_{S1} = X10\textcolor{red}{0}$ ,  $IN_1 = 011X$   
 $\Rightarrow OUT_1 = 010\textcolor{red}{X}$  ✓
  - $OUT_{S2} = X10\textcolor{red}{1}$ ,  $IN_1 = 011X$   
 $\Rightarrow OUT_2 = X11X$  ✗
  - $OUT_{S2} = X10\textcolor{red}{1}$ ,  $IN_1 = 0110$   
 $\Rightarrow OUT_2 = X11\textcolor{red}{1}$  ✓

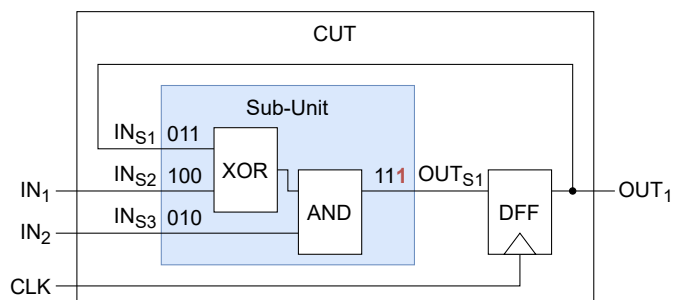


## Constrained Automatic Test Pattern Generation (ATPG) Test Pattern Retargeting

Strategy 3: use BMC (with sub-unit)

- DFF state is dependent on sub-circuit
- Initialization sequence is required  
 $DFF_{init} = 0$   
 $\Rightarrow IN_1 = X\ 100 \checkmark$   
 $\Rightarrow IN_2 = 0\ 010 \checkmark$   
 $\Rightarrow OUT_{S1} = 0\ 11\ \mathbf{1} \checkmark$
- Propagating fault from sub-circuit  
 $OUT_{S1} = 0\ 11\ \mathbf{1} \Rightarrow OUT_1 = X\ 011 \not\checkmark$

$IN_1 = X\ 100\ X, IN_2 = 0\ 010\ X$   
 $\Rightarrow OUT_{S1} = 0\ 11\ \mathbf{1}\ X \checkmark$   
 $\Rightarrow OUT_1 = X\ 011\ \mathbf{1} \checkmark$



## Constrained Automatic Test Pattern Generation (ATPG)

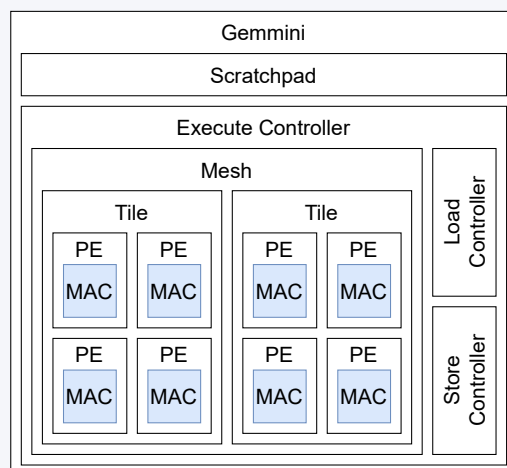
### Example: Gemini Mac-Unit SBST (Scale4Edge)

#### Testing of Systolic Array Architectures

Master Thesis – Christopher Dittrich

Create SBST to test multiply accumulate units (MAC)

- Run ATPG for multiply accumulate unit (MAC)
- Retarget patterns through hierarchy
  1. Retarget patterns to processing element (PE)
  2. Retarget patterns to tile
  3. Retarget patterns to mesh
  4. (Retarget patterns to mesh with delays)
  5. Retarget patterns to execute controller
- Convert execute controller patterns into SBST instructions



## Constrained Automatic Test Pattern Generation (ATPG)

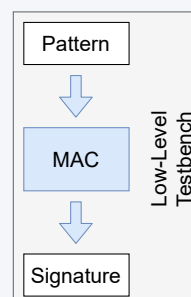
### Example: Gemmini Mac-Unit SBST (Scale4Edge)

Run ATPG for multiply accumulate unit

- Synthesized for Nangate 45nm Library with Yosys
- Combinational stuck-at ATPG using FreiTest

| ATPG Metric              | Result   |
|--------------------------|----------|
| Stuck-At Faults (unique) | 2,949    |
| Fault Coverage           | 99.15 %  |
| Fault Efficiency         | 100.00 % |
| Test Patterns            | 177      |
| Runtime                  | 4.68 s   |

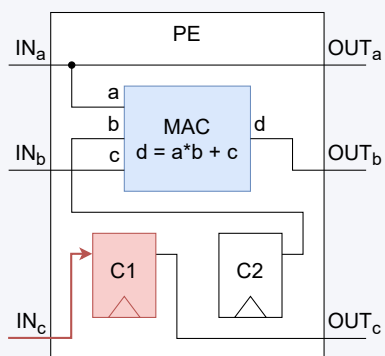
- Validation of patterns from FreiTest using testbench
  - Apply FreiTest test pattern to MAC unit
  - Check MAC unit outputs to be equal to FreiTest signature



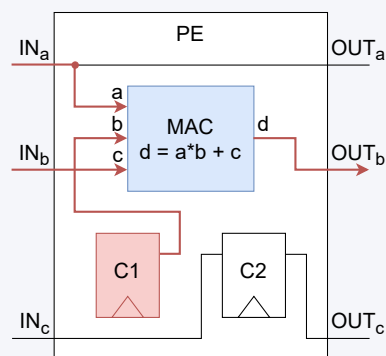
## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemini Mac-Unit SBST (Scale4Edge)

Retarget MAC patterns to processing element (PE) using strategy 1 (clever reasoning and intuition)



Initialize register C1:  
weight stationary, propagate mode



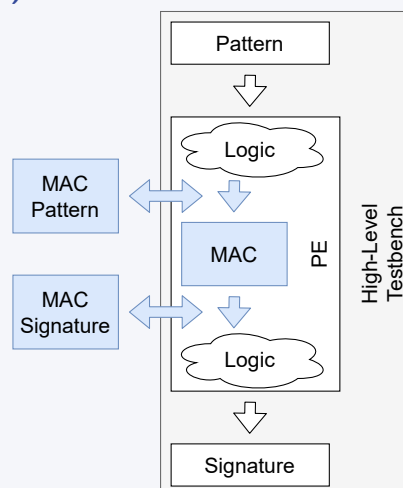
Apply pattern via C1:  
weight stationary, compute mode

## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemini Mac-Unit SBST (Scale4Edge)

Validate patterns for processing element (PE)

- Compare signatures to validate pattern application
  - Apply retargeted pattern to PE unit
  - Check PE unit outputs to be equal to expected signature
  - Check MAC unit inputs to be equal to MAC pattern
  - Check MAC unit outputs to be equal to MAC signature
- First timeframe is used for register initialization
- Second timeframe is relevant for MAC signature (test pattern application)

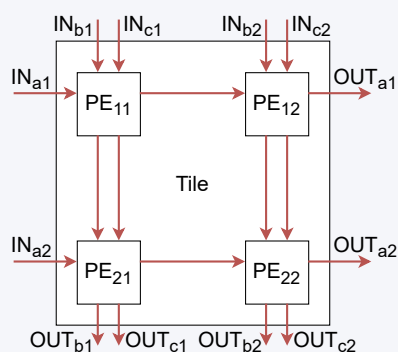


## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemini Mac-Unit SBST (Scale4Edge)

Retarget PE patterns to Tile using strategy 1 (trivial?)

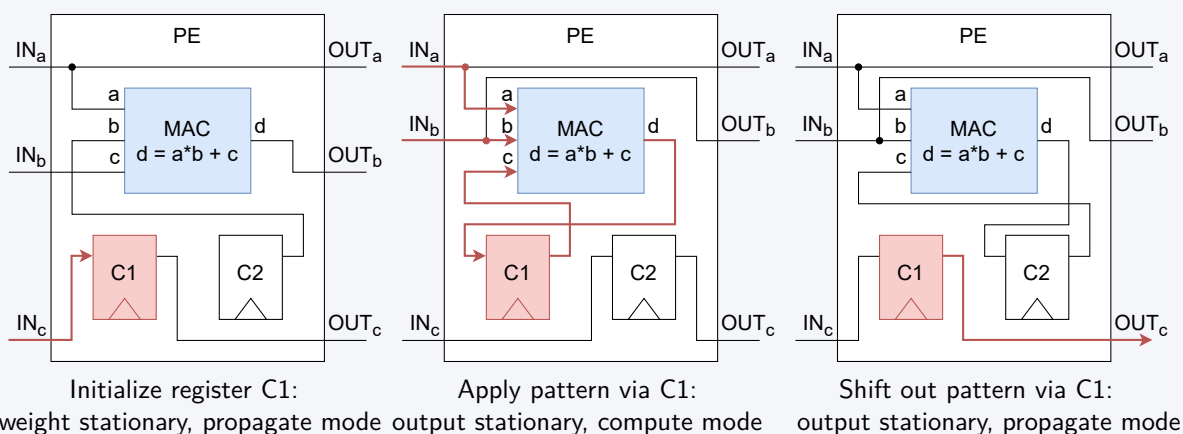
- Initialization sequence for registers C1 (for  $n$  rows):  
 For each row shift in pattern (requires  $n$  clock cycles)  
 Set  $IN_{c1} = IN_{c2} = PAT_{PE_c}$   
 $\Rightarrow$  finally  $\forall i \in \{11, 12, 21, 22\} : PE_{i, IN_c} = PAT_{PE_c} \checkmark$
- Apply pattern via registers C1 as before (single cycle)  
 Set  $IN_{a1} = IN_{a2} = PAT_{PE_a}$   
 $\Rightarrow \forall i \in \{11, 12, 21, 22\} : PE_{i, IN_a} = PAT_{PE_a} \checkmark$   
 Set  $IN_{b1} = IN_{b2} = PAT_{PE_b}$   
 $\Rightarrow PE_{11, IN_b} = PE_{12, IN_b} = PAT_{PE_b}$   
 $\Rightarrow PE_{21, IN_b} = PE_{22, IN_b} \neq PAT_{PE_b} \text{ ⚡}$ 
  - MAC output is connected to  $PE_{OUT_b}$
  - Output  $PE_{OUT_b}$  is connected to next input  $PE_{IN_b}$
  - Input  $PE_{IN_b}$  is different for  $PE_{21}$  and  $PE_{22}$



## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemini Mac-Unit SBST (Scale4Edge)

Retarget MAC patterns to processing element (PE) v2 using strategy 1

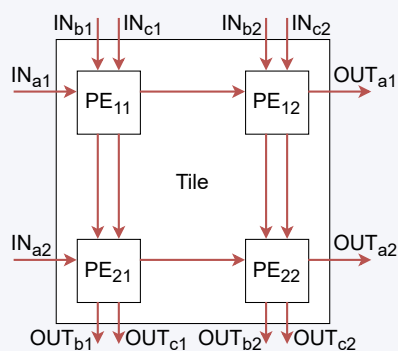


## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemini Mac-Unit SBST (Scale4Edge)

Retarget PE patterns to Tile using strategy 1 (again)

- Initialization sequence for registers C1 (for  $n$  rows):  
For each row shift in pattern (requires  $n$  clock cycles)  
Set  $IN_{c1} = IN_{c2} = PAT_{PE_c}$   
 $\Rightarrow$  finally  $\forall i \in \{11, 12, 21, 22\} : PE_{i, IN_c} = PAT_{PE_c}$  ✓
- Apply pattern via registers C1 (single cycle)  
Set  $IN_{a1} = IN_{a2} = PAT_{PE_a}$   
 $\Rightarrow \forall i \in \{11, 12, 21, 22\} : PE_{i, IN_a} = PAT_{PE_a}$  ✓  
Set  $IN_{b1} = IN_{b2} = PAT_{PE_b}$   
 $\Rightarrow \forall i \in \{11, 12, 21, 22\} : PE_{i, IN_b} = PAT_{PE_b}$  ✓
- Shift out pattern via registers C1 (for  $n$  rows):  
For each row shift in zeroes (requires  $n$  clock cycles) ✓
- Retargeted pattern validation via testbench like before  
(comparing inputs / outputs at PE units) ✓

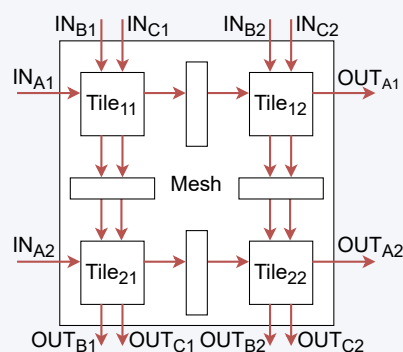


## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemmini Mac-Unit SBST (Scale4Edge)

Retarget Tile patterns to Mesh using strategy 1

- Mesh introduces pipeline registers
- Initialization sequence for registers C1 (for  $n$  rows):  
(Increase length by  $n - 1$  clock cycles) ✓
- Apply pattern via registers C1 (single cycle) ✓
- Shift out pattern via registers C1 (for  $n$  rows):  
(Increase length by  $n - 1$  clock cycles) ✓
- Retargeted pattern validation via testbench like before  
(comparing inputs / outputs at Tile units)
  - Mesh inputs for  $IN_{C1}$  and  $IN_{C2}$  are 2x8 bit
  - Tile inputs  $Tile_{IN_{C1}}$ ,  $Tile_{IN_{C2}}$  are 2x20 bits (zero extended) ⚡

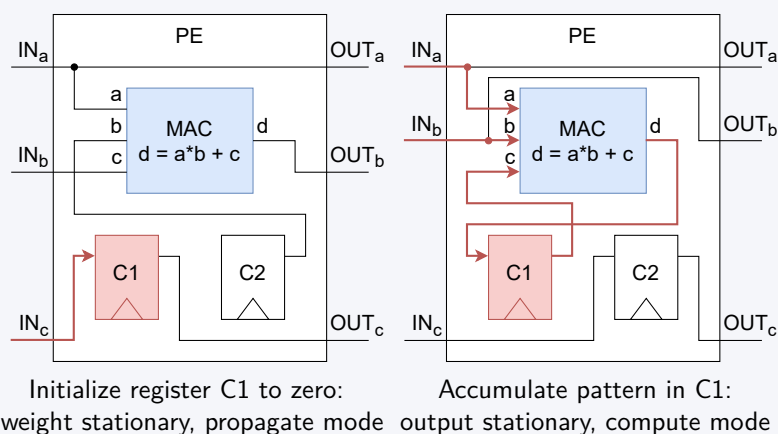


## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemini Mac-Unit SBST (Scale4Edge)

Retarget MAC patterns to processing element (PE) v3 using strategy 1

1. Set C1 to zero ( $IN_c = 00_{16}$ )  
 $\Rightarrow C1 = 00000_{16}$
2. Accumulate pattern in C1  
 (e.g.,  $PAT_{MAC_c} = 04321_{16}$ )  
 Set  $IN_a = 7F_{16}$ ,  $IN_b = 7F_{16}$   
 $\Rightarrow C1 = 03F01_{16}$   
 Set  $IN_a = 18_{16}$ ,  $IN_b = 2C_{16}$   
 $\Rightarrow C1 = 04321_{16}$
3. Apply test pattern  
 $IN_a = PAT_{MAC_a}$   
 $IN_b = PAT_{MAC_b}$
4. Shift out result



## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemmini Mac-Unit SBST (Scale4Edge)

Retarget MAC patterns to PE, to Tile, and to Mesh using strategy 1

- Initialization sequence for register C1 (for  $n$  rows):
  - Zero register C1 via  $IN_{C1} = 00_{16}$  for  $n$  cycles (weight stationary, propagate mode)
  - Accumulate pattern in register C1 via  $IN_A, IN_B$  (output stationary, compute mode)
  - $\Rightarrow C1 = PAT_{MAC_c}$  ✓
- Apply pattern via register C1 (single cycle, output stationary, compute mode)
  - $\Rightarrow MAC_a = IN_A = PAT_{MAC_a}$  ✓
  - $\Rightarrow MAC_b = IN_B = PAT_{MAC_b}$  ✓
  - $\Rightarrow MAC_c = C1 = PAT_{MAC_c}$  ✓
- Shift out pattern via registers C1 (for  $n$  rows):
  - Set  $IN_{C1} = 00_{16}$  for  $n$  cycles (output stationary, propagate mode)
  - $\Rightarrow$  finally  $OUT_c = MAC_d$  ✓
- Retargeted pattern validation via testbench like before (comparing inputs / outputs at Tile units) ✓

Constrained Automatic Test Pattern Generation (ATPG)  
Example: Gemmini Mac-Unit SBST (Scale4Edge)

Retarget Mesh patterns to execute controller using strategy 1

- Initialize accelerator for test
  - Apply output stationary mode via CONFIG command
  - Build A, B matrices of  $n \times n$  that accumulate C1 and apply the test pattern
  - Build C matrix of  $n \times n$  zero values
  - Store A, B, and C matrices in scratchpad memory
- Zero C1, accumulate in C1, apply pattern and shift out result
  - Apply C1 zero matrix via PRELOAD and COMPUTE\_AND\_FLIP commands
  - Apply accumulate and test matrices via COMPUTE\_AND\_STAY commands
- Retargeted pattern validation via testbench like before (comparing inputs / outputs at Mesh units) ✓

|                                            |
|--------------------------------------------|
| Configure<br>Gemmini                       |
| Write Matrix<br>A, B, C<br>into Scratchpad |
| Matrix<br>Multiplication<br>$C := C + A*B$ |
| Matrix<br>Multiplication<br>$C := C + A*B$ |
| ...                                        |
| Read C from<br>Scratchpad                  |

Constrained Automatic Test Pattern Generation (ATPG)  
Example: Gemini Mac-Unit SBST (Scale4Edge)

Convert execute controller patterns into SBST instructions

- CONFIG command  $\Rightarrow$  gemmini\_config\_ex
- PRELOAD command  $\Rightarrow$  gemmini\_preload\_zeros
- COMPUTE\_AND\_FLIP command  $\Rightarrow$  gemmini\_compute\_preloaded
- COMPUTE\_AND\_STAY command  $\Rightarrow$  gemmini\_compute\_accumulated
- Build A, B matrices of  $n \times n$  that accumulate C1 and apply the test pattern:
- Extract factors from patterns  $\Rightarrow$  export as static program data

$\Rightarrow$  A one to one mapping of all controller commands to C code possible

|                                            |
|--------------------------------------------|
| Configure<br>Gemmini                       |
| Write Matrix<br>A, B, C<br>into Scratchpad |
| Matrix<br>Multiplication<br>C := C + A*B   |
| Matrix<br>Multiplication<br>C := C + A*B   |
| ...                                        |
| Read C from<br>Scratchpad                  |

## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemmini Mac-Unit SBST (Scale4Edge)

```
int sbist_gemmini(const pattern_t& pattern) {
    // Storage for pattern expanded to DIM x DIM MAC units
    // in form of DIM x DIM matrices.
    elem_t A_IN[DIM][DIM] row_align(1);
    elem_t B_IN[DIM][DIM] row_align(1);
    acc_t C_OUT[DIM][DIM] row_align(1);

    // Configure accelerator
    gemmini_flush(0);
    gemmini_config_ld(DIM * sizeof(elem_t));
    gemmini_config_st(DIM * sizeof(acc_t));
    gemmini_config_ex(OUTPUT_STATIONARY, NO_ACTIVATION, 0);
    ...
}
```

## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemmini Mac-Unit SBST (Scale4Edge)

```
...
// Prefill scratchpad for A, B and C. If there are more factors than DIM,
// then multiple matrices (convolutions) are required and applied after each other.
for (size_t convolution = 0u; convolution < pattern.convolutions; convolution++) {
    for (size_t i = 0u; i < DIM; i++) {
        for (size_t j = 0u; j < DIM; j++) {
            // Factors for accumulation and test extracted from patterns.
            A_IN[i][j] = pattern.a[convolution*DIM + j];
            B_IN[i][j] = pattern.b[convolution*DIM + i];
        }
    }
    // Copy expanded matrices into scratchpad
    gemmini_mvin(A_IN, SCRATCH_A + convolution*DIM);
    gemmini_mvin(B_IN, SCRATCH_B + convolution*DIM);
}
gemmini_preload_zeros(SCRATCH_C);
...
```

## Constrained Automatic Test Pattern Generation (ATPG)

### Example: Gemmini Mac-Unit SBST (Scale4Edge)

```
...
// Execute the test pattern on all MAC units in parallel
for (size_t convolution = 0u; convolution < pattern.convolutions; convolution++) {
    if (convolution == 0)
        gemmini_compute_preloaded(SCRATCH_A + convolution*DIM, SCRATCH_B + convolution*DIM);
    else
        gemmini_compute_accumulated(SCRATCH_A + convolution*DIM, SCRATCH_B + convolution*DIM);
}

// Validate result of each MAC unit
gemmini_mvout(C_OUT, SCRATCH_C);
for (size_t i = 0u; i < DIM; i++)
    for (size_t j = 0u; j < DIM; j++)
        if (C_OUT[i][j] != pattern.expected)
            return SBST_FAULT;
return SBST_OKAY;
}
```

## Constrained Automatic Test Pattern Generation

### Example: Gemmini Mac-Unit SBST (Scale4Edge)

- Gemmini accelerator with  $16 \times 16$  8-bit input, 20-bit output MACs
- Synthesized for Nangate 45nm Library with Yosys
- Only stuck-at faults in MAC units targeted



| SBST Metric              | Result   | SBST Segment                      | Result             |
|--------------------------|----------|-----------------------------------|--------------------|
| MAC units                | 256      | RISC-V Code (.text)               | 312 B              |
| Stuck-At Faults (unique) | 754,944  |                                   | (101 Instructions) |
| Fault Coverage           | 99.15 %  | Static Data (.rodata)             | 11,936 B           |
| Fault Efficiency         | 100.00 % | Static Data (.rodata, compressed) | 2,877 B            |
| Patterns                 | 177      |                                   |                    |
| Matrices (Convolutions)  | 240      |                                   |                    |
| Matrices per Pattern     | 1.36     |                                   |                    |

**Testing of Systolic Array Architectures**  
*Master Thesis – Christopher Dittrich*

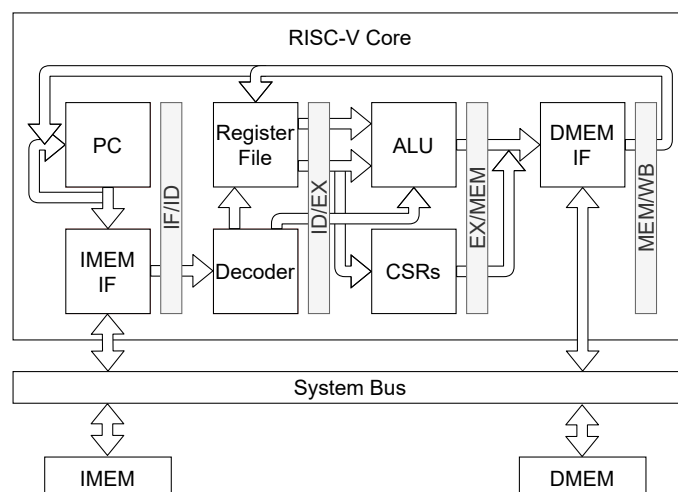
## Constrained Automatic Test Pattern Generation (ATPG) - References I

- [1] [Andreas Riefert et al.](#) “An effective approach to automatic functional processor test generation for small-delay faults”. In: *2014 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 2014, pp. 1–6. DOI: [10.7873/DATE.2014.140](#).
- [2] [Andreas Riefert et al.](#) “Effective generation and evaluation of diagnostic SBST programs”. In: *2016 IEEE 34th VLSI Test Symposium (VTS)*. 2016, pp. 1–6. DOI: [10.1109/VTS.2016.7477279](#).

## Software-based Self-Test (SBST)

### Software-based Self-Test (SBST)

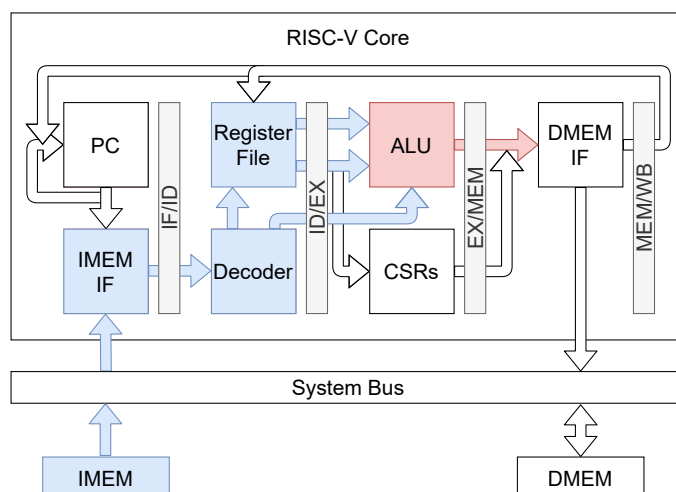
- Processor needs testing too
  - Processor can generate patterns
  - Processor applies patterns to itself
- Instructions in memory are patterns
  - Decoder expands test patterns
  - Operations and operands are applied to execution units (e.g., ALU)



## Software-based Self-Test (SBST)

### Software-based Self-Test (SBST)

- Processor needs testing too
  - Processor can generate patterns
  - Processor applies patterns to itself
- Instructions in memory are patterns
  - Decoder expands test patterns
  - Operations and operands are applied to execution units (e.g., ALU)



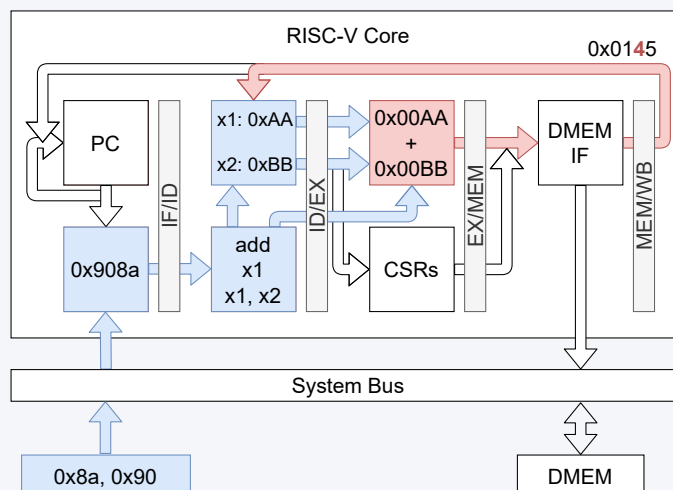
## Software-based Self-Test (SBST)

### Example: SBST Instruction Sequence

Test for stuck-at 0 at ALU output bit 7

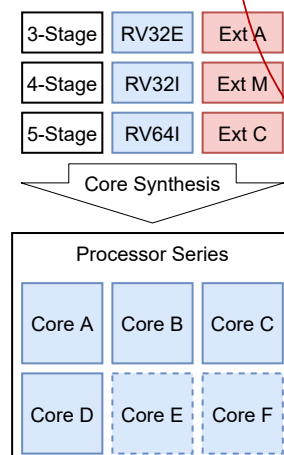
```
li x1, 0xAA
li x2, 0xBB
add x1, x1, x2
```

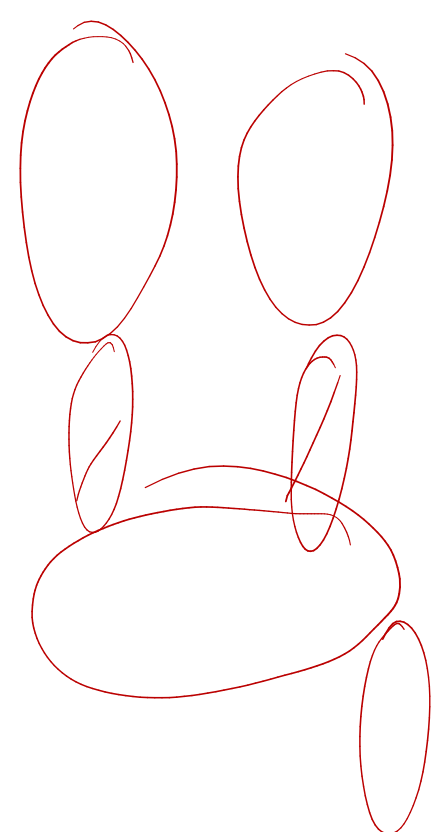
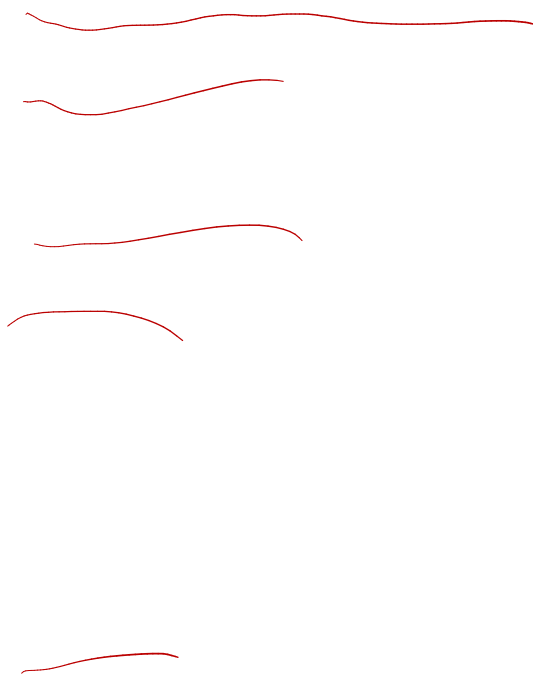
- IMEM:  $8a_{16}, 90_{16} \Rightarrow 908a_{16}$
- Decoder: **add x1, x1, x2**  
 Operation            +  
 Destination (rd)    x1  
 Sources (rs1, rs2) x1, x2
- Register File:  $AA_{16}$  (rs1),  $BB_{16}$  (rs2)
- ALU:  $00AA_{16} + 00BB_{16} = 0145_{16}$   
 (expected  $0165_{16}$ )



## Software-based Self-Test (SBST)

- Synthesis of custom processors via generator frameworks
    - Configurable pipeline stages: IF ID EX (MEM) (WB)
    - Configurable base ISA: RV32E, RV32I, RV64I
    - Configurable extensions
      - A (Atomics)
      - F (Single Floating Point)
      - M (Multiplication)
      - D (Double Floating Point)
      - Custom extensions
      - C (Compressed)
  - Many processor series from many vendors (DarkRISCV, PicoRV32, CV32E40P, Ibex, MINRES, ...)
- ⇒ Plethora of synthesized processor variants for each series



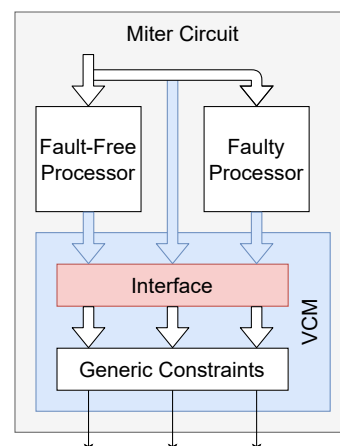


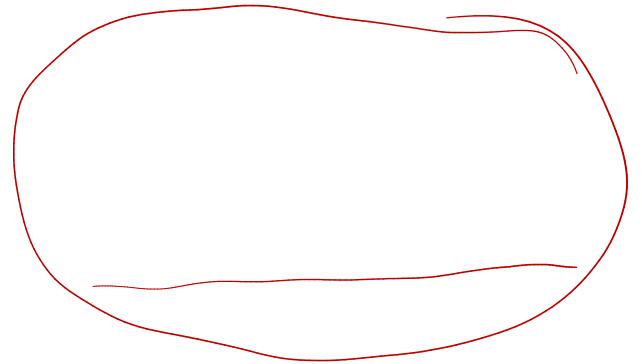
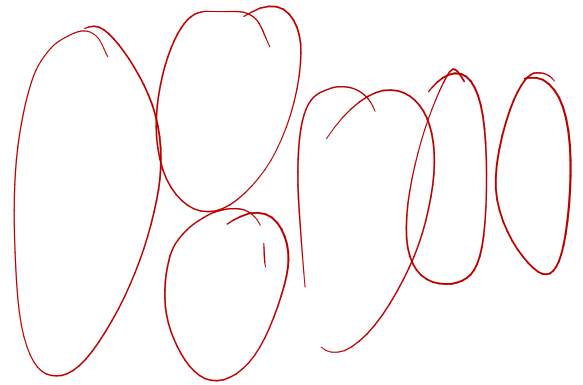
## Software-based Self-Test (SBST)

### Generic Constraints via Validity Checker Module (VCM)

- VCM observes fault-free and faulty version of ...
  - Processor and internal pipeline state
  - Register file contents
  - Instruction bus transactions
  - Data bus transactions
- VCM is tightly coupled to processor-specific signals
  - Constraints are not directly portable
  - VCM has to be changed for each new variant

⇒ A well-defined interface is required





## Software-based Self-Test (SBST)

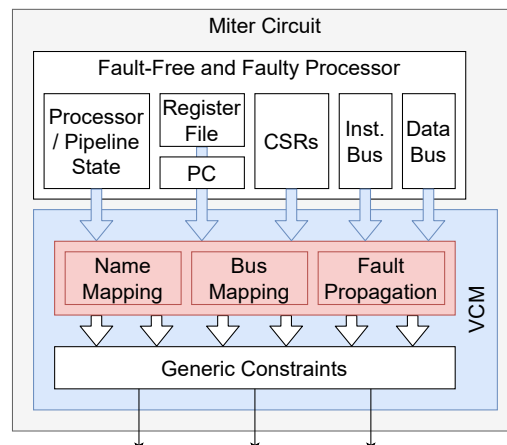
### Generic Constraints via Validity Checker Module (VCM)

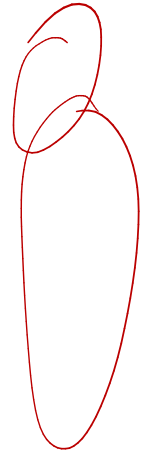
- Mapping layer [1] adapts processor to VCM interface

- Processor and pipeline state (running, bubble, flush)
- Architecture registers (contents)
- Program counter (contents)
- Instruction bus transactions (read, write, idle)
- Data bus transactions (read, write, idle)
- Control and status registers (CSR) (read, write)
- Fault propagation (registers, instruction / data bus)

- Generic constraint set is applied to the interface

- SBST behavior constraints
- Environment constraints
- ATPG constraints
- Propagation constraints





$x_1 \ x_i$

## Software-based Self-Test (SBST) Example: VCM Processor Signal Mapping

```
module vcm(
  input CLK,
  input RES,
  input HLT,
  ...
);
generic_vcm constraints(
  .clock(CLK),
  .processor_reset(RES),
  .processor_halt(HLT),
  .processor_run(!RES && !HLT),
  ...
);
endmodule
```

```
module vcm(
  input [31:0] \PC_reg[{index}]/Q ,
  input [31:0] \REGS_reg[1][{index}]/Q ,
  input [31:0] \REGS_reg[2][{index}]/Q ,
  ...
);
generic_vcm constraints(
  .register_pc(\PC_reg[{index}]/Q ),
  .register_x1(\REGS_reg[1][{index}]/Q ),
  .register_x2(\REGS_reg[2][{index}]/Q ),
  ...
);
endmodule
```

*Handwritten red notes:*  
 $\{ \text{register\_pc, register\_x1, register\_x2} \}$   
 $\{ \text{PC, REGS\_reg[1], REGS\_reg[2]} \}$

0

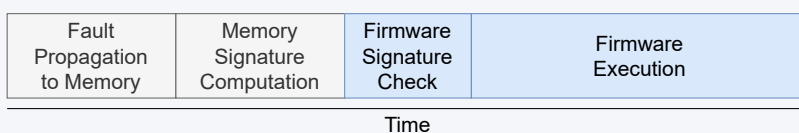
CRC HASH

## Software-based Self-Test (SBST) Example: Startup SBST Generation (Scale4Edge)



Towards SAT-based SBST generation for RISC-V cores  
IEEE LATS 2021 – Poster

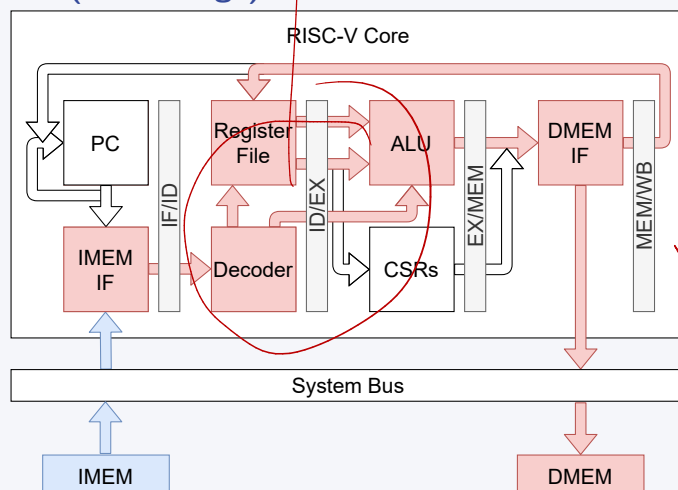
- SBST designed to run on system startup (POST)
  - Runs before firmware initialization
  - Data memory contains no firmware data yet⇒ SBST can use data memory for fault propagation
- SBST propagates faults to data memory
  - SBST computes signature in data memory
  - Firmware checks signature during initialization

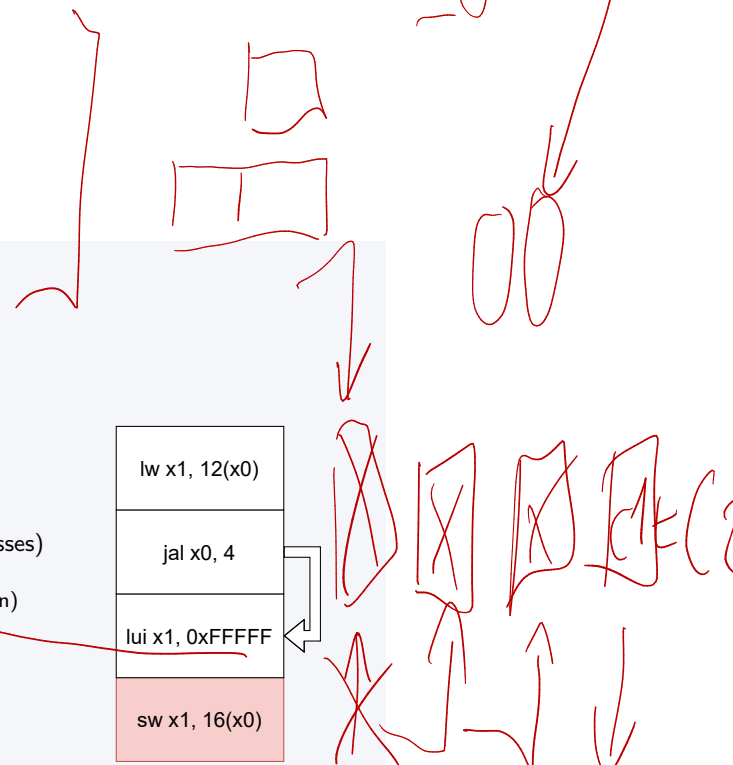
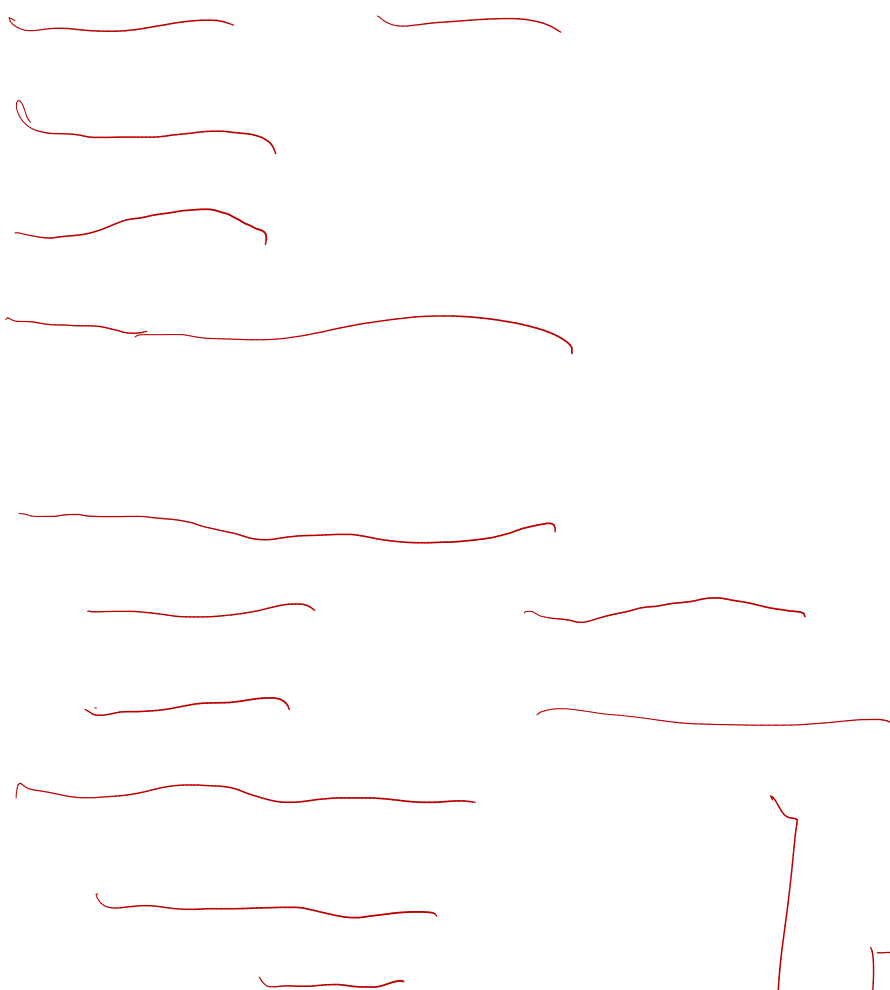
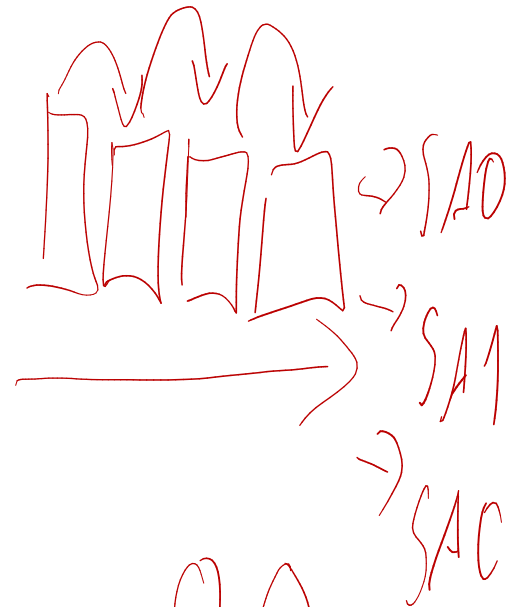


## Software-based Self-Test (SBST)

### Example: Startup SBST Generation (Scale4Edge)

- Instruction memory contains patterns
- Patterns propagate through processor
- Detects faults on the propagation path
  - Instruction Memory Interface
  - Instruction Decoder
  - Register file
  - Arithmetic Logic Unit
  - Data Memory Interface
- Fault propagation to data memory

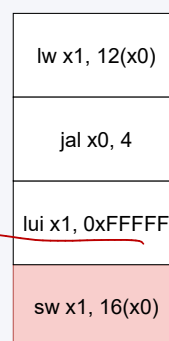




## Software-based Self-Test (SBST)

### Example: Startup SBST Generation (Scale4Edge)

- Use constrained ATPG with generic VCM architecture
  - Processor running, not stalled
  - Only valid RISC-V instructions are allowed
  - CSRs reads and writes are disallowed
  - Force linear program flow (always continue with next instruction)
  - Force linear addressed of data memory (no overlapping memory accesses)
    - Never read the same memory cell (unique values per cell)
    - Never write the same memory cell (no fault propagation is overwritten)
  - Fault propagation on data memory
    - Difference in written value (value  $AA_{16}$  vs value  $BB_{16}$ )
    - Difference in write mask (mask  $0001_2$  vs mask  $1111_2$ )
    - Difference in write flag (write  $1_2$  vs write  $0_2$ )
- Extract SBST instructions from instruction memory transactions
  - Concatenate all SBST instructions into single SBST program
  - ⇒ Result is a linear SBST program consisting of RISC-V instructions



Software-based Self-Test (SBST)  
Example: Startup SBST Generation (Scale4Edge)

- Open-Source DarkRISCv and PicoRV32 RISC-V processor cores
- Synthesized for Nangate 45nm Library with Yosys

| Processor core  | Fault Coverage                   | Fault Efficiency | Generated Sequences | Program Instructions | Generation Time |
|-----------------|----------------------------------|------------------|---------------------|----------------------|-----------------|
| DarkRISCv RV32E | 82.90 %                          | 83.00 %          | 944                 | 3,809                | 1.60 h          |
| DarkRISCv RV32I | 89.90 %                          | 89.38 %          | 1,832               | 7,218                | 2.95 h          |
| PicoRV32 RV32I  | 79.55 %                          | 79.92 %          | 2,121               | 8,927                | 13.54 h         |
| PicoRV32 RV32IM | – exhausted solver time limits – |                  |                     |                      | ≥ 27.78 h       |

0000001

R := R XOR

XOR

## Software-based Self-Test (SBST) Example: Online SBST Generation (Scale4Edge)

Scale4Edge

RISC-V®

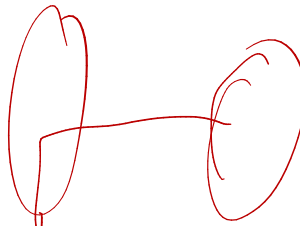
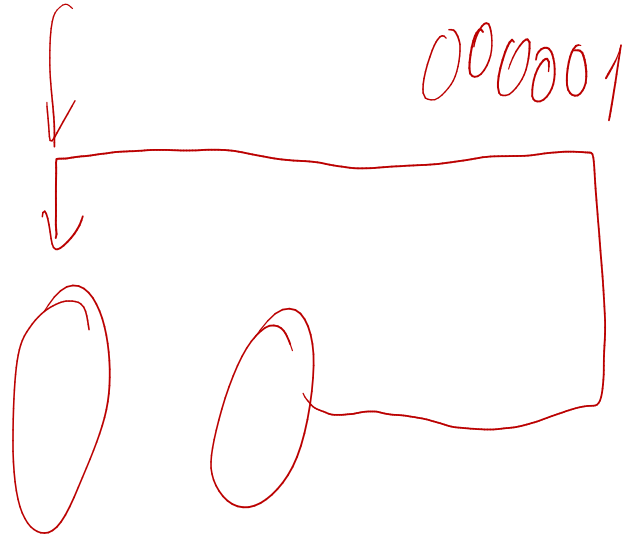
### Constraint-Based Automatic SBST Generation for RISC-V Processor Families

IEEE ETS 2023 – Paper

- SBST designed to run on during system idle
  - Runs whenever firmware is idle (free compute resources)
  - System state can not be modified (minimal assumptions about firmware)
- ⇒ SBST builds a checksum in a selected register (checksum register)
- SBST propagates faults to checksum register
  - SBST uses XOR / XORI instructions to update checksum
  - Firmware checks checksum after leaving system idle

|                                     |                                |                       |                                     |                                |                       |
|-------------------------------------|--------------------------------|-----------------------|-------------------------------------|--------------------------------|-----------------------|
| Fault<br>Propagation<br>to Register | Firmware<br>Signature<br>Check | Firmware<br>Execution | Fault<br>Propagation<br>to Register | Firmware<br>Signature<br>Check | Firmware<br>Execution |
|-------------------------------------|--------------------------------|-----------------------|-------------------------------------|--------------------------------|-----------------------|

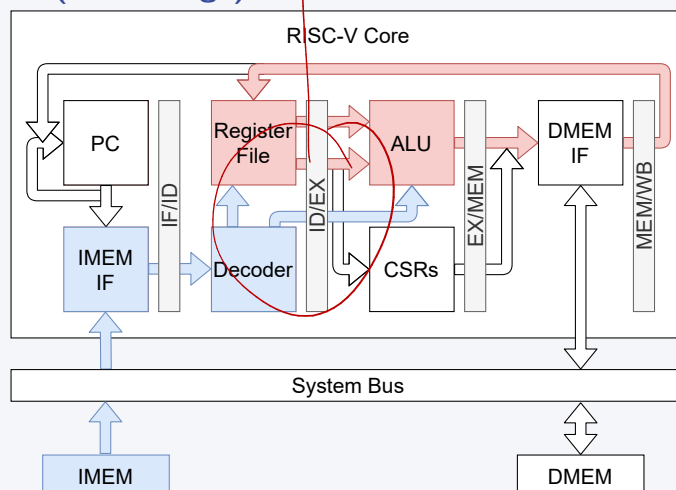
Time

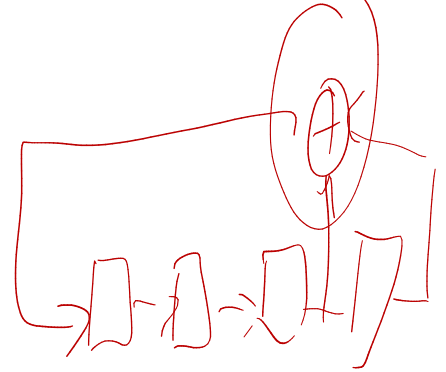


## Software-based Self-Test (SBST)

### Example: Online SBST Generation (Scale4Edge)

- Instruction memory contains patterns
- Patterns propagate through processor
- Target faults on essential units
  - Register file
  - Arithmetic Logic Unit
- Fault propagation to checksum register





0

0

0

0

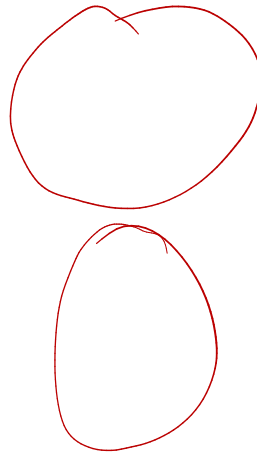
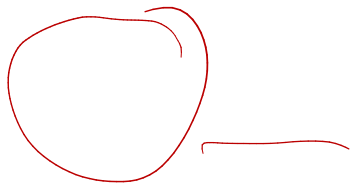
0

## Software-based Self-Test (SBST)

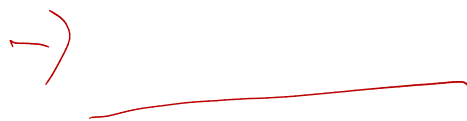
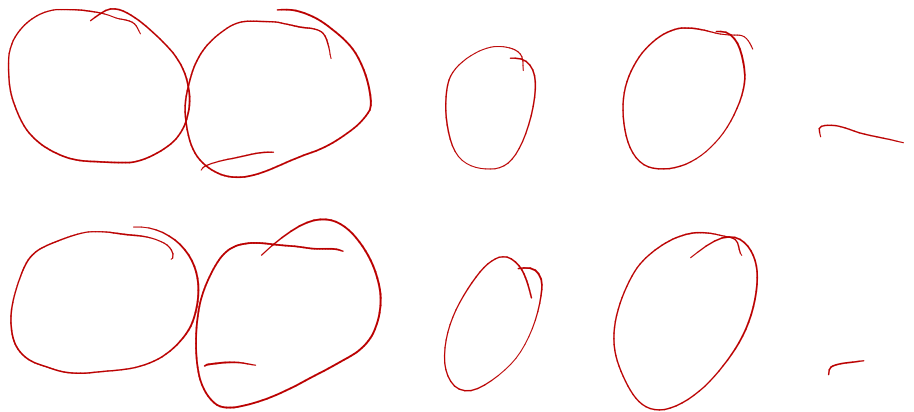
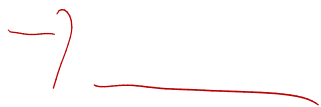
### Example: Online SBST Generation (Scale4Edge)

- Use constrained ATPG with generic VCM architecture
  - Processor running, not stalled
  - Only arithmetic instructions allowed (only linear program flow)
  - No CSR accesses (no modification of system state)
  - No access to data memory (no modification of firmware state)
  - Fault propagation to checksum register (e.g., x1)
    - Only XOR / XORI instructions allowed for checksum register
    - Exactly one source (rs1, rs2) and destination (rd) are checksum register
    - Rotate register after fault propagation (reduces fault cancellations)
    - Emulates Cyclic Redundancy Check (CRC) or Multiple Input Signature Register (MISR)
- Extract SBST instructions from instruction memory transactions
  - Concatenate all SBST instructions into single SBST program
  - ⇒ Result is a linear SBST program consisting of RISC-V instructions

|                  |
|------------------|
| lui x2, 0xFFFFF  |
| lui x3, 0xFFFFF  |
| add x2, x2, x3   |
| xor x1, x1, x2   |
| rotr x1, x1, 0x1 |



Freitest

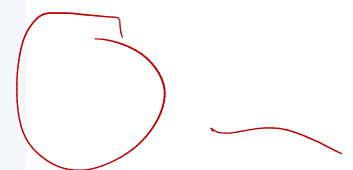


## Software-based Self-Test (SBST)

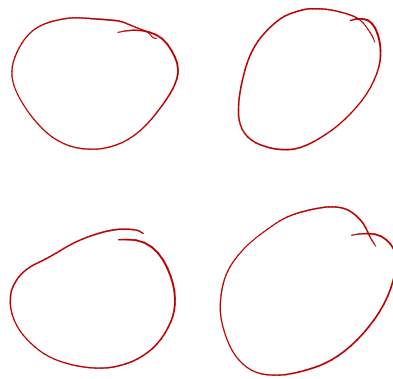
### Example: Online SBST Generation (Scale4Edge)

- Proprietary Scale4Edge RISC-V processor core
- Synthesized for Nangate 45nm Library with Synopsys DC

|                                | Fault Coverage | Test Coverage  | Generated Sequences | Program Instructions | Generation Time |
|--------------------------------|----------------|----------------|---------------------|----------------------|-----------------|
| Instruction Fetch Stage        | 6.54 %         | 14.01 %        | 13                  | 76                   | 15.59 h         |
| Instruction Decode Stage       | 40.29 %        | 45.26 %        | 58                  | 479                  | 4.08 h          |
| ↳ <b>Register File</b>         | <b>93.40 %</b> | <b>99.67 %</b> | <b>730</b>          | <b>7,196</b>         | <b>34.19 h</b>  |
| Execute Stage                  | 11.66 %        | 13.65 %        | 67                  | 514                  | 29.82 h         |
| ↳ <b>Arithmetic Logic Unit</b> | <b>99.70 %</b> | <b>99.89 %</b> | <b>428</b>          | <b>3,087</b>         | <b>5.36 h</b>   |
| Memory Stage                   | 12.88 %        | 18.94 %        | 17                  | 94                   | 2.45 h          |
| Writeback Stage                | 25.63 %        | 26.67 %        | 13                  | 59                   | 0.84 h          |
| Miscellaneous                  | 4.27 %         | 5.21 %         | 15                  | 84                   | 4.06 h          |
| Sum                            | 45.40 %        | 51.06 %        | 1,341               | 11,589               | 96.39 h         |



701X



## Software-based Self-Test (SBST)

### Example: Online SBST Generation (Scale4Edge)

- Proprietary Scale4Edge RISC-V processor core
- Synthesized for Nangate 45nm Library with Synopsys DC

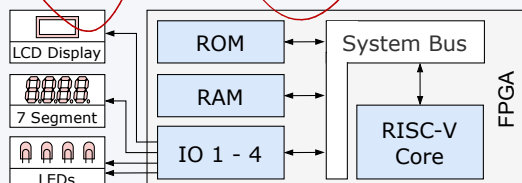
|                                | Detected       | Undetected    | End Not Reached | Data Bus Used | Exception Occured | Untestable    |
|--------------------------------|----------------|---------------|-----------------|---------------|-------------------|---------------|
| Instruction Fetch Stage        | 3.43 %         | 78.16 %       | 1.46 %          | 0.01 %        | 7.22 %            | 6.20 %        |
| Instruction Decode Stage       | 40.57 %        | 70.67 %       | 0.91 %          | 0.09 %        | 7.44 %            | 4.30 %        |
| └ <b>Register File</b>         | <b>91.23 %</b> | <b>0.12 %</b> | 0.00 %          | 0.00 %        | <b>0.08 %</b>     | <b>5.24 %</b> |
| Execute Stage                  | 11.69 %        | 75.24 %       | 0.24 %          | 0.00 %        | 1.13 %            | 5.55 %        |
| └ <b>Arithmetic Logic Unit</b> | <b>93.75 %</b> | <b>0.03 %</b> | 0.00 %          | 0.00 %        | <b>5.89 %</b>     | <b>0.12 %</b> |
| Memory Stage                   | 14.42 %        | 76.57 %       | 1.05 %          | 0.00 %        | 0.12 %            | 5.64 %        |
| Writeback Stage                | 23.88 %        | 73.69 %       | 0.68 %          | 0.00 %        | 0.05 %            | 0.29 %        |
| Miscellaneous                  | 4.25 %         | 83.03 %       | 0.37 %          | 0.03 %        | 7.65 %            | 3.85 %        |
| Sum                            | 38.51 %        | 48.86 %       | 0.51 %          | 0.01 %        | 3.34 %            | 4.83 %        |

## Software-based Self-Test (SBST) Example: Online SBST Generation (Scale4Edge)

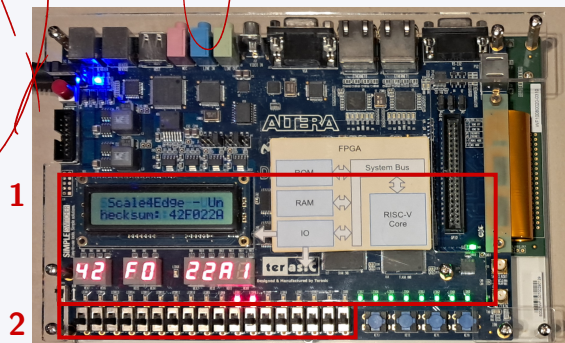
### FPGA-based Online-SBST Demonstrator

Scale4Edge Roadshow

- Proprietary Scale4Edge RISC-V processor core
- Altera DE2-115 dev. board (Cyclone IV FPGA)
- Stuck-at SBST is run periodically by firmware
  1. Display and LEDs visualize SBST state
  2. Switches allow for stuck-at fault injection



Scale4Edge  
RISC-V®



## Software-based Self-Test (SBST) - References I

- [1] Tobias Faller et al. "Constraint-Based Automatic SBST Generation for RISC-V Processor Families". In: *2023 IEEE European Test Symposium (ETS)*. 2023, pp. 1–6. DOI: [10.1109/ETS56758.2023.10174156](https://doi.org/10.1109/ETS56758.2023.10174156).
- [2] Tobias Faller et al. "Towards SAT-Based SBST Generation for RISC-V Cores". In: *2021 IEEE 22nd Latin American Test Symposium (LATS)*. 2021, pp. 1–2. DOI: [10.1109/LATS53581.2021.9651819](https://doi.org/10.1109/LATS53581.2021.9651819).
- [3] Tobias Faller et al. "Special Session: Software-Based Self-Test Generation for RISC-V – Stuck-At Generation, Functional Cell-Aware Untestability, and FPGA Demonstration –". In: *2024 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. 2024, pp. 1–6. DOI: [10.1109/DFT63277.2024.10753557](https://doi.org/10.1109/DFT63277.2024.10753557).

